THE RECONSILIATION ALGORITHM: DIFFING VIRTUAL TREES

Created: 2025-12-22 Mon 14:59

THIS CHAPTER COVERS

THIS CHAPTER COVERS

- Comparing two virtual DOM trees

THIS CHAPTER COVERS

- Comparing two virtual DOM trees
- Finding the differences between two objects

THIS CHAPTER COVERS

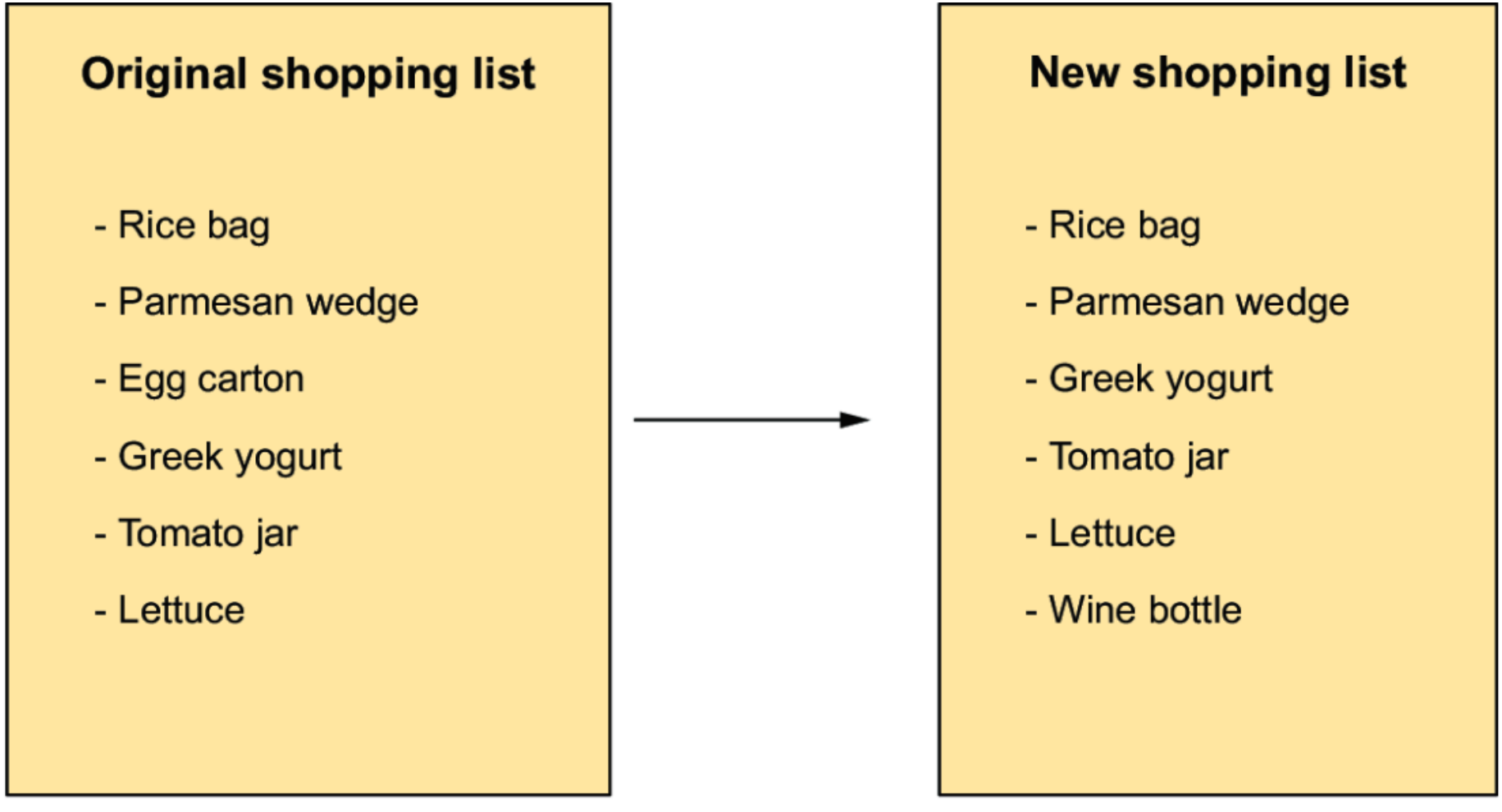
- Comparing two virtual DOM trees
- Finding the differences between two objects
- Finding the differences between two arrays

THIS CHAPTER COVERS

- Comparing two virtual DOM trees
- Finding the differences between two objects
- Finding the differences between two arrays
- Finding a sequence of operations that transforms one array into another.

HOW SHOULD WE APPROACH THE PROBLEM

The analogy is similar to updating a shopping list



Two options:

Two options:

- You can start over:

Two options:

- You can start over:
  - emptying the cart by putting back the items and then picking up the items in the updated list.

Two options:

- You can start over:
  - emptying the cart by putting back the items and then picking up the items in the updated list.
- Compare the two lists and figure out which items were removed and which items were added.

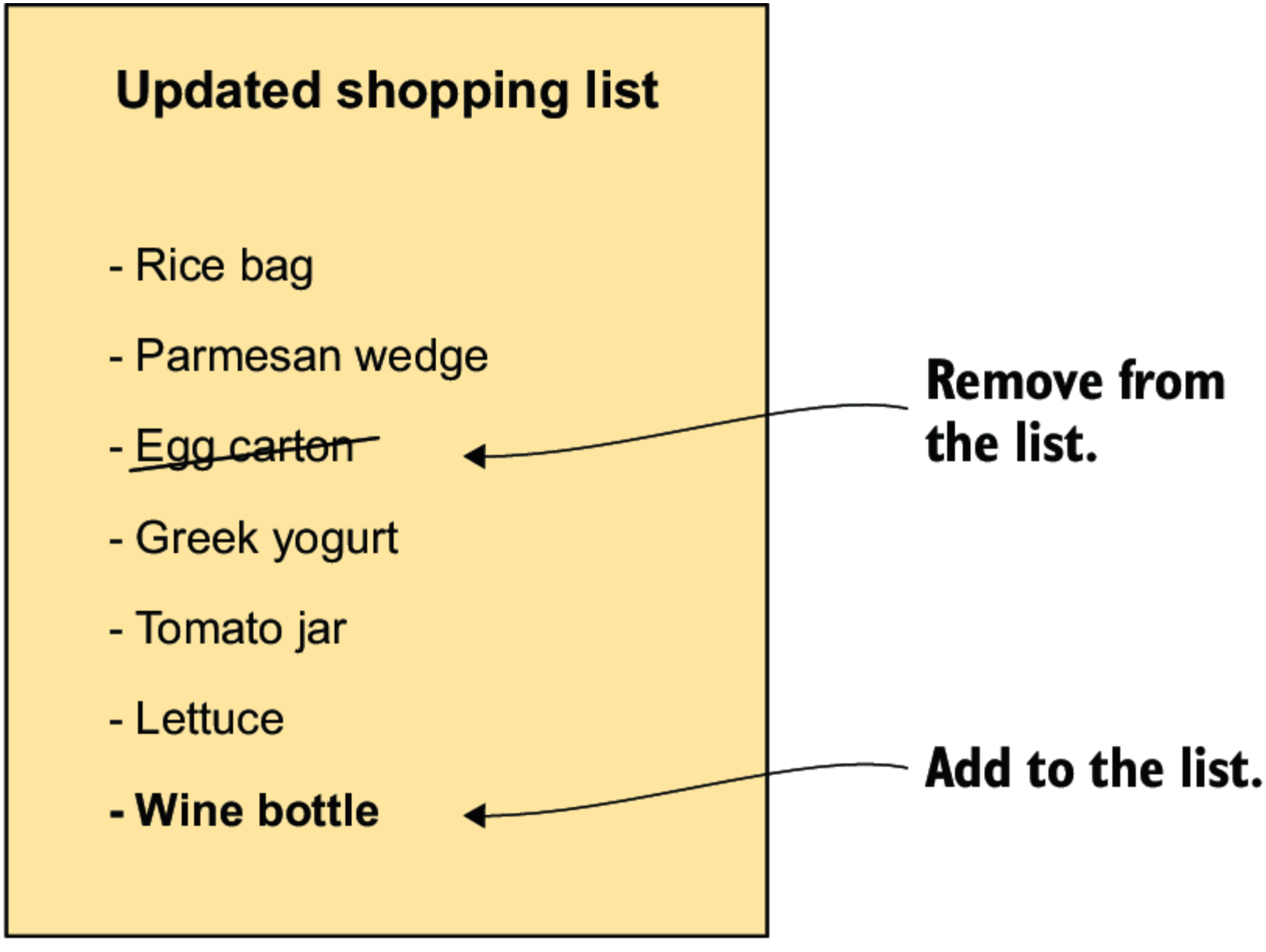
Two options:

- You can start over:
  - emptying the cart by putting back the items and then picking up the items in the updated list.
- Compare the two lists and figure out which items were removed and which items were added.
  - Get rid of the items that were removed from the original list

Two options:

- You can start over:
  - emptying the cart by putting back the items and then picking up the items in the updated list.
- Compare the two lists and figure out which items were removed and which items were added.
  - Get rid of the items that were removed from the original list
  - Pick up the items that were added to the updated list.

- The ideal variant is to choose the second option.


Updated shopping list
- Rice bag
- Parmesan wedge
- Egg carton
- Greek yogurt
- Tomato jar
- Lettuce
- Wine bottle
Remove from the list.
Add to the list.

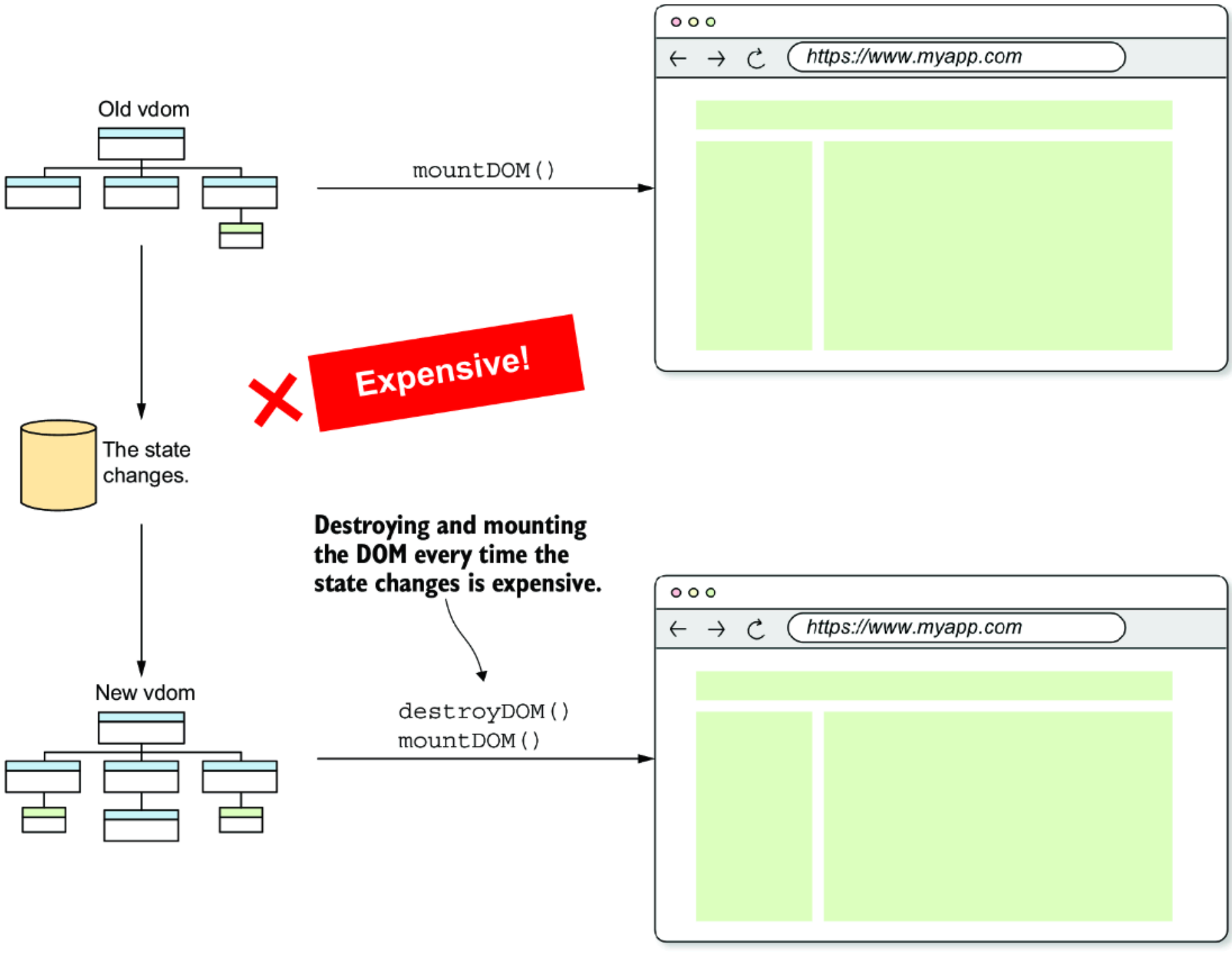
- compare two virtual Document Object Model (DOM) trees

- compare two virtual Document Object Model (DOM) trees
- figure out the changes

- compare two virtual Document Object Model (DOM) trees
- figure out the changes
- apply the differences to the real DOM.

- compare two virtual Document Object Model (DOM) trees
- figure out the changes
- apply the differences to the real DOM.
- The framework must be smart enough to figure out the operations to apply to the real DOM

HOW WE CURRENTLY MANAGE THE APPLICATION


Old vdom
mountDOM()
https://www.myapp.com
Expensive!
The state changes.
Destroying and mounting the DOM every time the state changes is expensive.
New vdom
destroyDOM()
mountDOM()
https://www.myapp.com

OBJECTIVE

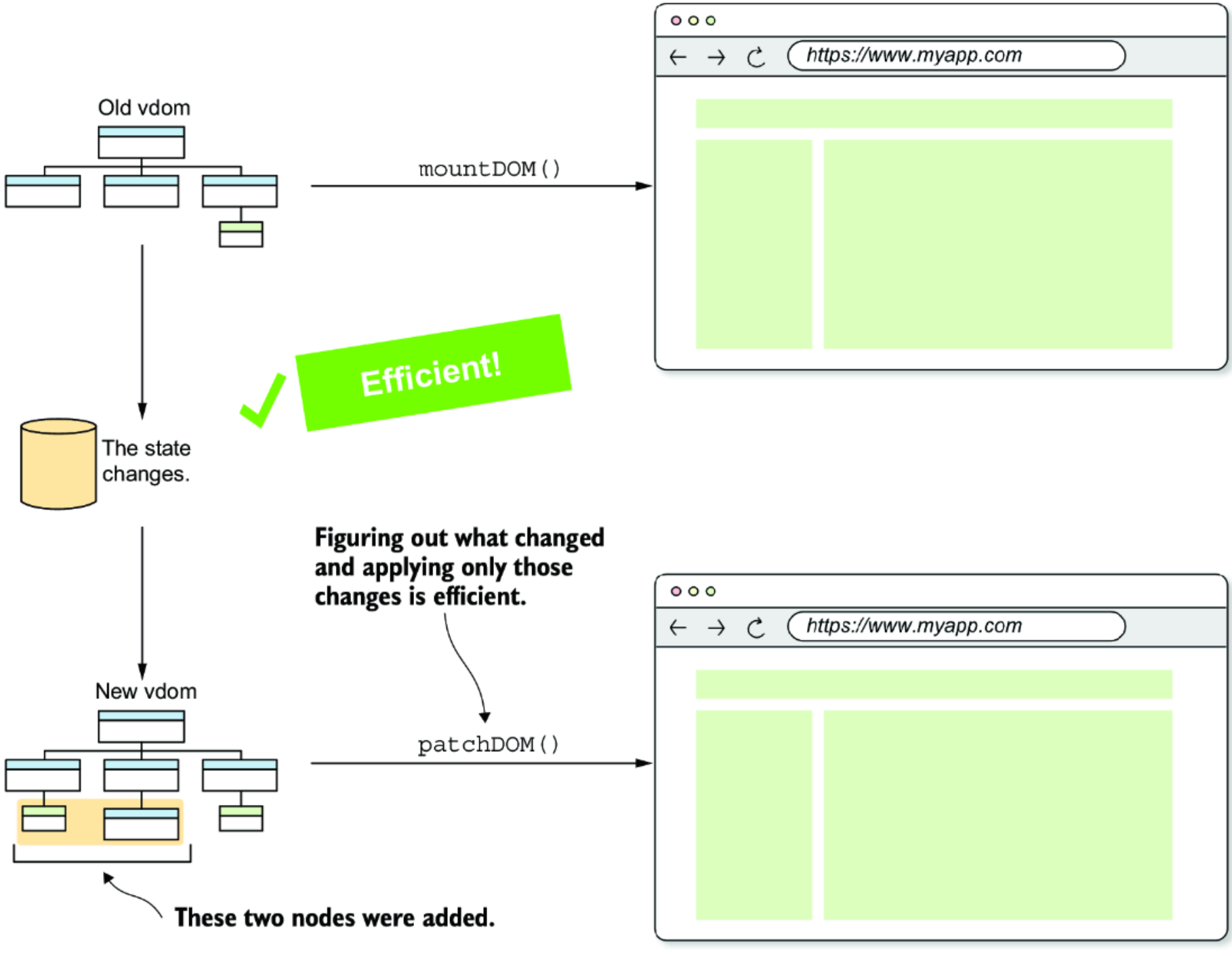
OBJECTIVE

- Figure out the differences between two virtual DOM trees

OBJECTIVE

- Figure out the differences between two virtual DOM trees
- Apply those differences to the real DOM so that the framework can update it more efficiently

OBJECTIVE


Old vdom
mountDOM()
https://www.myapp.com
Efficient!
The state changes.
Figuring out what changed and applying only those changes is efficient.
New vdom
patchDOM()
https://www.myapp.com
These two nodes were added.

THE RECONCILIATION ALGORITHM

THE RECONCILIATION ALGORITHM

Can be reduced to three functions:

THE RECONCILIATION ALGORITHM

THE RECONCILIATION ALGORITHM

- Function to find the differences between two objects and return:

THE RECONCILIATION ALGORITHM

- Function to find the differences between two objects and return:
  - the keys that were added,

THE RECONCILIATION ALGORITHM

- Function to find the differences between two objects and return:
  - the keys that were added,
  - the keys that were removed

THE RECONCILIATION ALGORITHM

- Function to find the differences between two objects and return:
  - the keys that were added,
  - the keys that were removed
  - the keys whose associated values changed

THE RECONCILIATION ALGORITHM

THE RECONCILIATION ALGORITHM

- A function to find the differences between two arrays, returning:

THE RECONCILIATION ALGORITHM

- A function to find the differences between two arrays, returning:
  - the items that were added

THE RECONCILIATION ALGORITHM

- A function to find the differences between two arrays, returning:
  - the items that were added
  - the items that were removed

THE RECONCILIATION ALGORITHM

THE RECONCILIATION ALGORITHM

- Function that, given two arrays, figures out a sequence of operations to apply to the first array and transform it into the second array

COMPARING TWO VIRTUAL DOM TREES

- The view is a function of the state:
  - For state changes, the virtual DOM representing the view changes.
- The reconciliation algorithm compares the old vDOM with the new virtual DOM after state changes.
  - *reconcile* the two virtual DOM trees!
  - figure out what changed and apply those changes to the real DOM.

To compare two virtual DOM trees

- start comparing the two root nodes, checking whether they’re equal

To compare two virtual DOM trees

- start comparing the two root nodes, checking whether they're equal
  - also check if their attributes or event listeners have changed.

COMPARING TWO VIRTUAL DOM TREES

If changes are detected:

COMPARING TWO VIRTUAL DOM TREES

If changes are detected:

- destroy the subtree rooted at the old node and replace it with the new node and everything under it.

COMPARING TWO VIRTUAL DOM TREES

If changes are detected:

- destroy the subtree rooted at the old node and replace it with the new node and everything under it.
- Then compare the children of the two nodes, traversing recursively in a depth-first manner.

- When comparing two arrays of child nodes:

- When comparing two arrays of child nodes:
  - Figure out whether a node was added, removed, or moved to a different position.

- When comparing two arrays of child nodes:
  - Figure out whether a node was added, removed, or moved to a different position.
- Depth-first traversal is the natural order in which the DOM is modified.

DFS traversal ensures:

DFS traversal ensures:

- Changes are applied to a complete branch of the tree before moving to the next branch.

DFS traversal ensures:

- Changes are applied to a complete branch of the tree before moving to the next branch.
- Traversing this way is important for *fragments*:

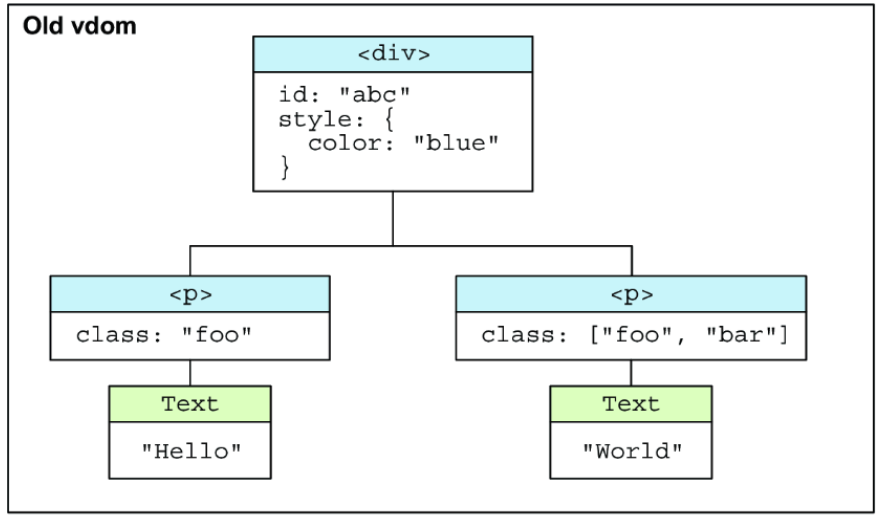
DFS traversal ensures:

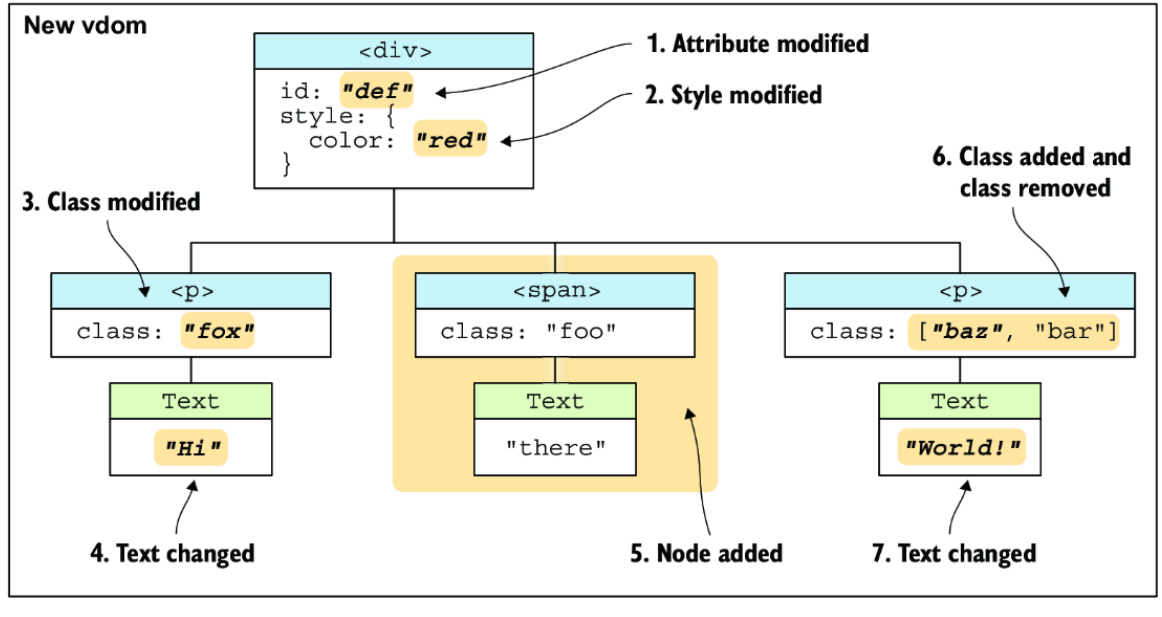
- Changes are applied to a complete branch of the tree before moving to the next branch.
- Traversing this way is important for *fragments*:
  - The children of a fragment are added to the fragment's parent

DFS traversal ensures:

- Changes are applied to a complete branch of the tree before moving to the next branch.
- Traversing this way is important for *fragments*:
  - The children of a fragment are added to the fragment's parent
  - If the number of children changes, the change could potentially alter the indices where the siblings of the fragment's parent are inserted.

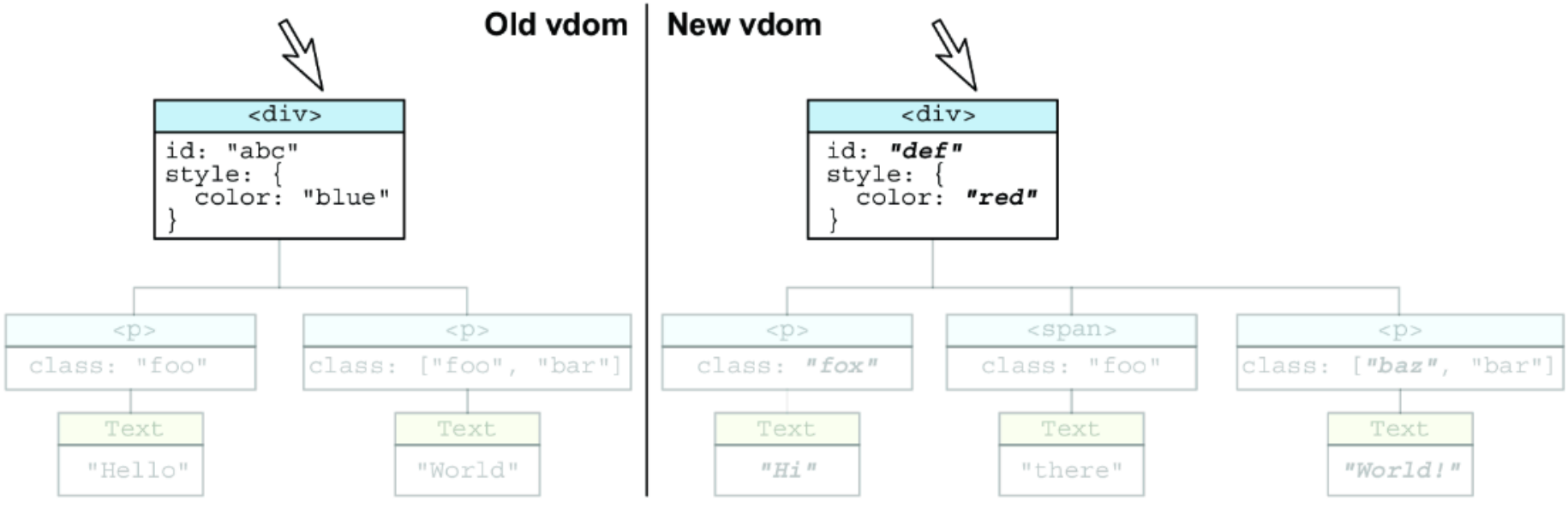
FINDING THE DIFFERENCES





STEP 1 - COMPARE THE ROOT NODES

- compare the two root `<div>` nodes
  - the `id` attribute changed from `"abc"` to `"def"`
  - the `style` attribute changed from `"color: blue"` to `"color: red"`.


Old vdom
New vdom

id: "abc"
style: {
color: "blue"
}
<p>
class: "foo"
Text
"Hello"
<p>
class: ["foo", "bar"]
Text
"World"

id: "def"
style: {
color: "red"
}
<p>
class: "fox"
Text
"Hi"

class: "foo"
Text
"there"
<p>
class: ["baz", "bar"]
Text
"World!"

STEP 2 - COMPARING THE CHILDREN

STEP 2 - COMPARING THE CHILDREN

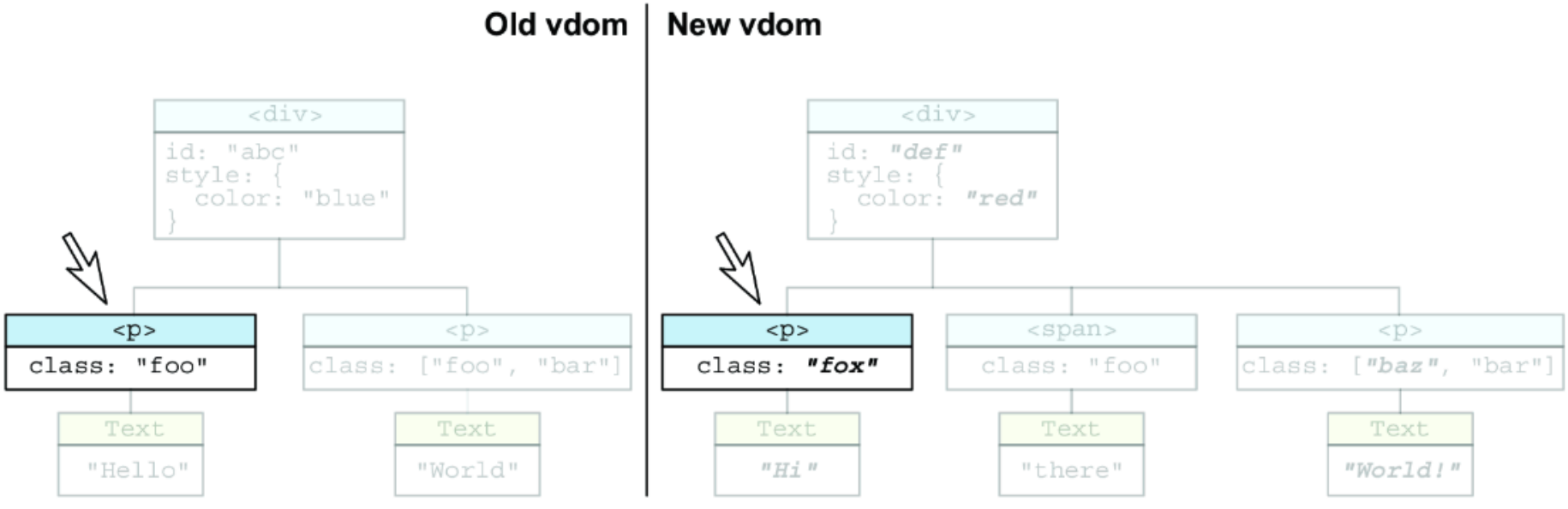
- Compare the two children of the `<div>` node one by one.

STEP 2 - COMPARING THE CHILDREN

- Compare the two children of the `<div>` node one by one.
- The `<p>` element is in the same position in both trees;

STEP 2 - COMPARING THE CHILDREN

- Compare the two children of the `<div>` node one by one.
- The `<p>` element is in the same position in both trees;
- The `class` attribute changed from `"foo"` to `"fox"`.


Old vdom
New vdom

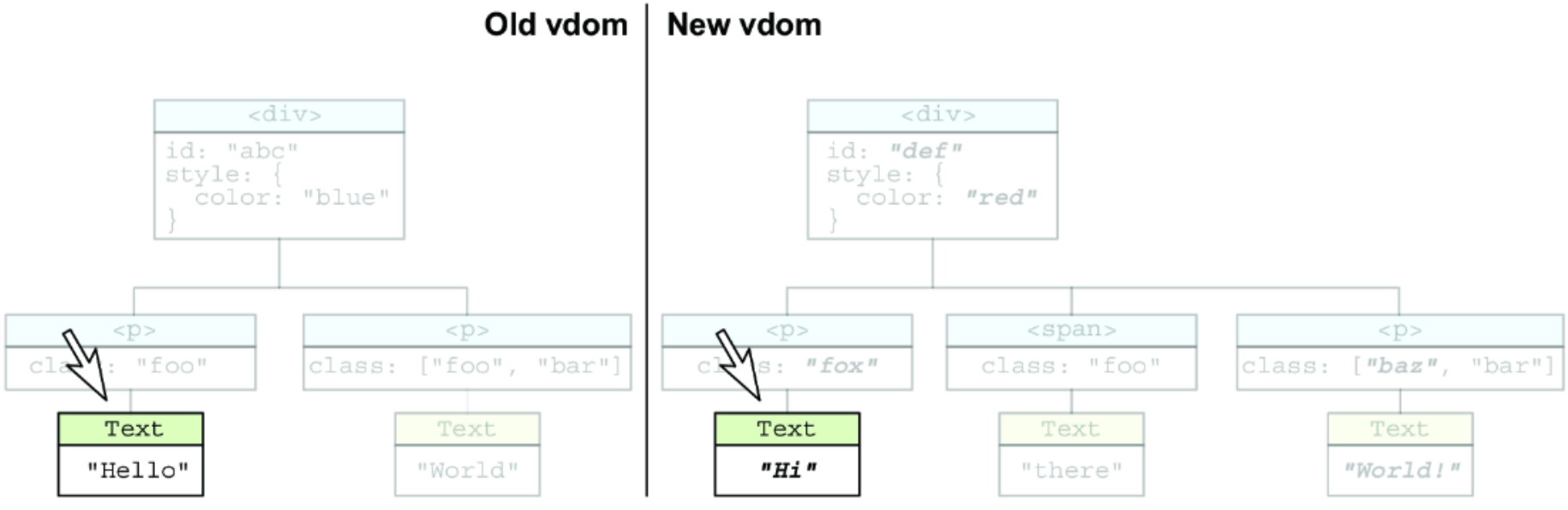
id: "abc"
style: {
color: "blue"
}
<p>
class: "foo"
<p>
class: ["foo", "bar"]
Text
"Hello"
Text
"World"

id: "def"
style: {
color: "red"
}
<p>
class: "fox"

class: "foo"
<p>
class: ["baz", "bar"]
Text
"Hi"
Text
"there"
Text
"World!"

STEP 3 - GRANDCHILD FROM THE FIRST CHILD

- Compare the children of the <p> element:
  - the text content has changed from `"Hello"` to `"Hi"`


Old vdom
New vdom

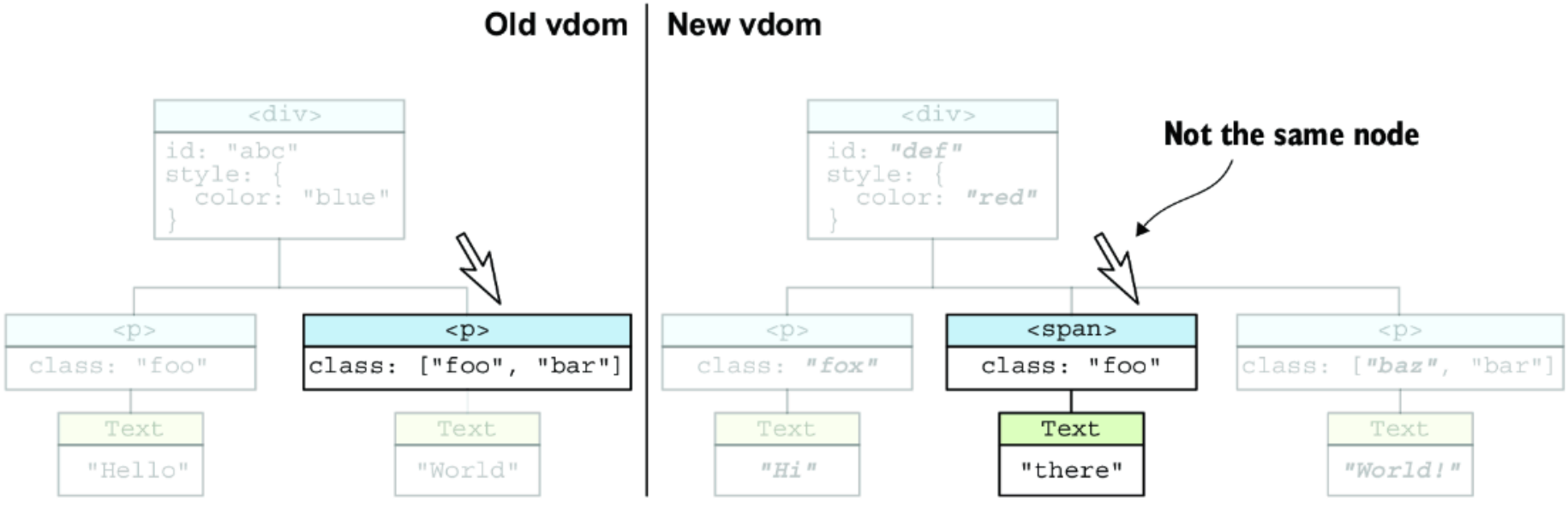
id: "abc"
style: {
color: "blue"
}
<p>
class: "foo"
Text
"Hello"
<p>
class: ["foo", "bar"]
Text
"World"

id: "def"
style: {
color: "red"
}
<p>
class: "fox"
Text
"Hi"

class: "foo"
Text
"there"
<p>
class: ["baz", "bar"]
Text
"World!"

STEP 4 - SECOND CHILD

- The second child was a <p>, now it’s a
- The <p> has moved one position to the right.


Old vdom
New vdom

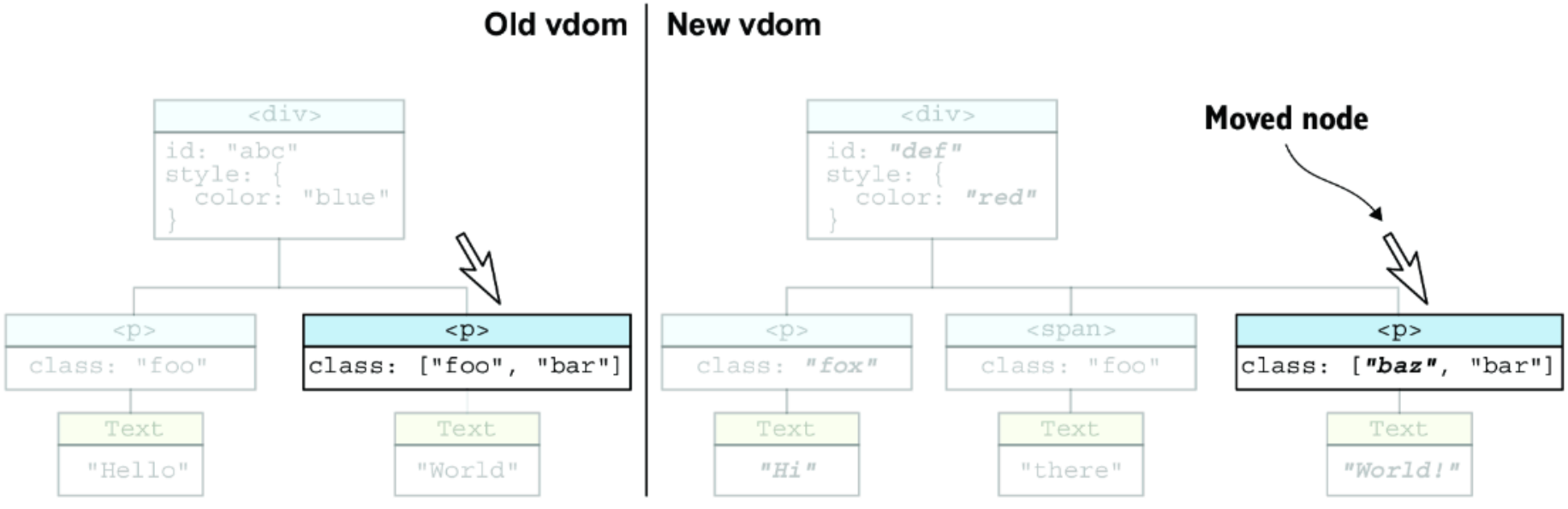
id: "abc"
style: {
color: "blue"
}
<p>
class: "foo"
Text
"Hello"
<p>
class: ["foo", "bar"]
Text
"World"

id: "def"
style: {
color: "red"
}
Not the same node
<p>
class: "fox"
Text
"Hi"

class: "foo"
Text
"there"
<p>
class: ["baz", "bar"]
Text
"World!"

STEP 5 - THIRD CHILD

- <p> node that you know moved one position
  - `class` attribute changed from `["foo","bar"]` to `["baz", "bar"]`,


Old vdom
New vdom

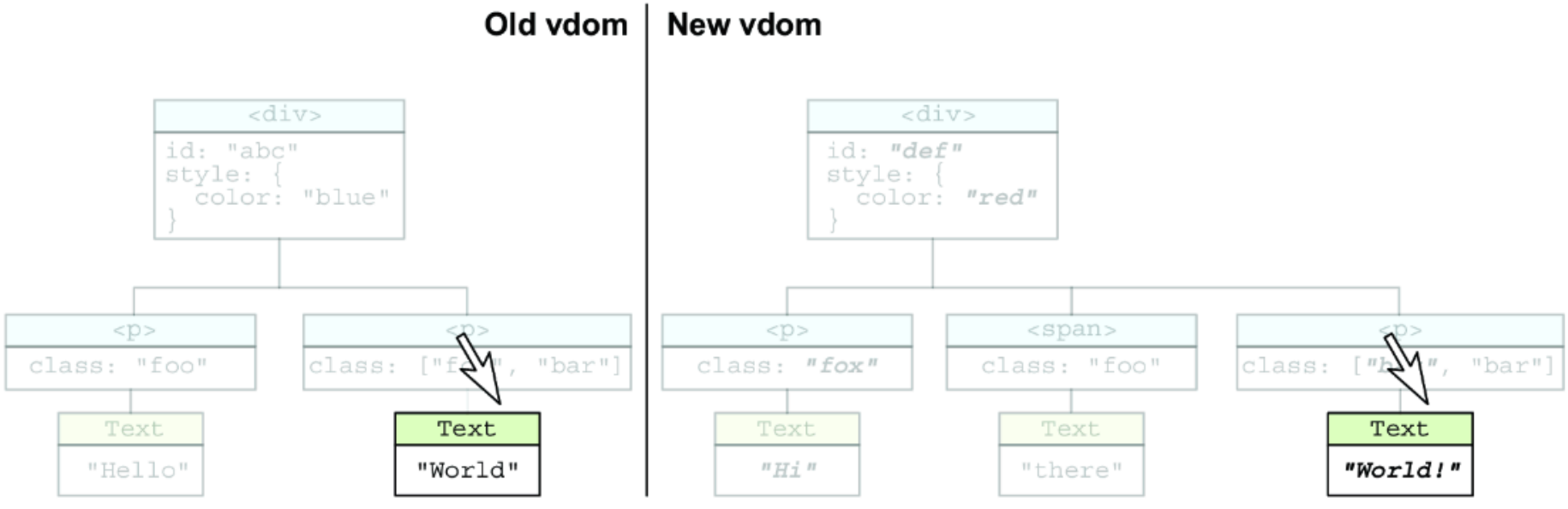
id: "abc"
style: {
color: "blue"
}
<p>
class: "foo"
Text
"Hello"
<p>
class: ["foo", "bar"]
Text
"World"

id: "def"
style: {
color: "red"
}
<p>
class: "fox"
Text
"Hi"

class: "foo"
Text
"there"
Moved node
<p>
class: ["baz", "bar"]
Text
"World!"

STEP 6 - GRANDCHILD FROM THE THRID CHILD

- The child of the `<p>` element has the text content changed from `"World"` to `"World!"`.


Old vdom
New vdom

id: "abc"
style: {
color: "blue"
}
<p>
class: "foo"
Text
"Hello"
<p>
class: ["f
", "bar"]
Text
"World"

id: "def"
style: {
color: "red"
}
<p>
class: "fox"
Text
"Hi"

class: "foo"
Text
"there"
<p>
class: ["h
", "bar"]
Text
"World!"

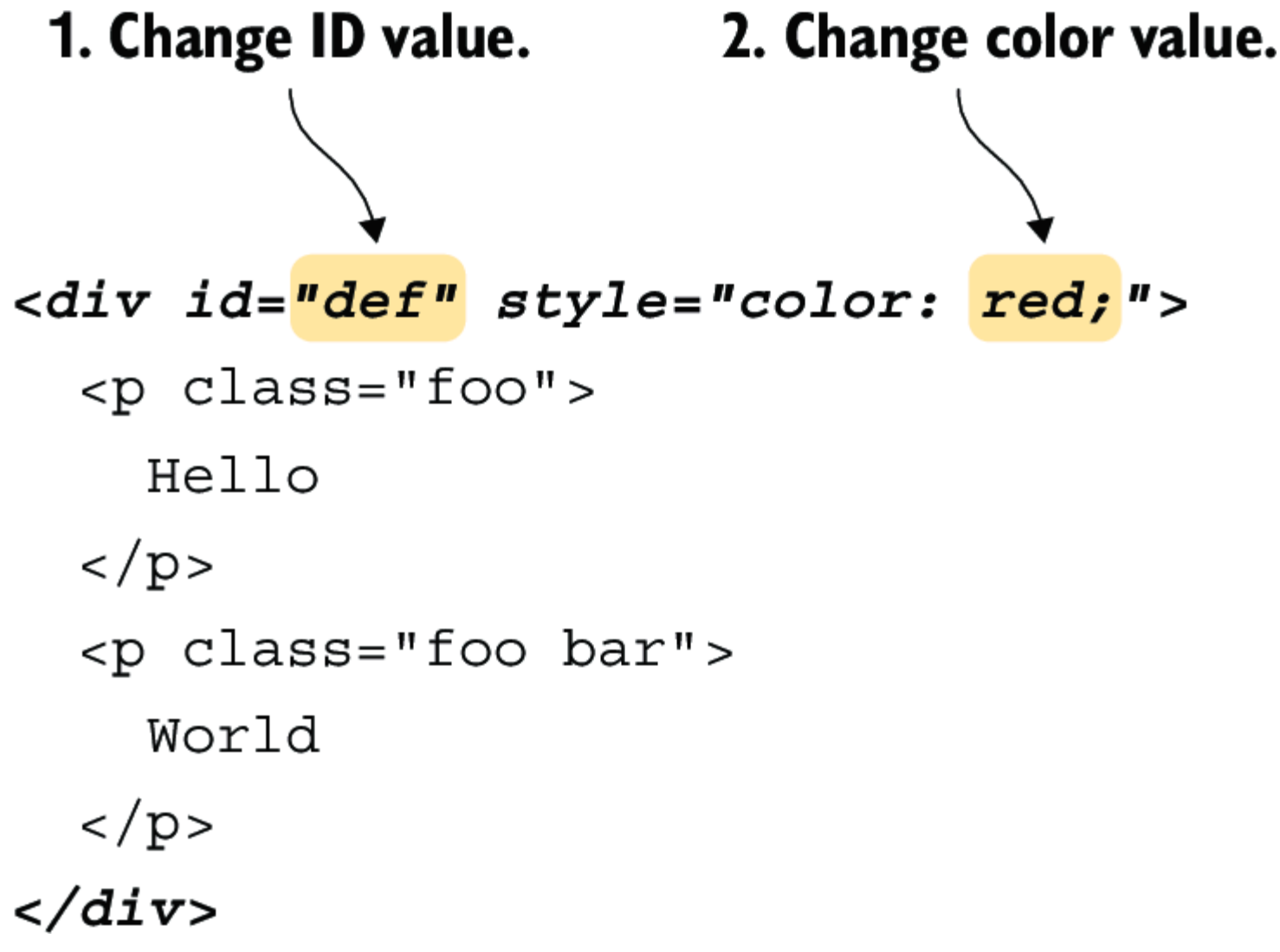
- a total of seven changes that need to be applied to the real DOM.

EXAMPLE - APPLYING THE CHANGES

```
<div id="abc" style="color: blue;">
  <p class="foo">
    Hello
  </p>
  <p class="foo bar">
    World
  </p>
</div>
```

MODIFYING THE ID AND STYLE ATTRIBUTES OF THE PARENT <DIV> NODE

```
div.id = 'def'
div.style.color = 'red'
```

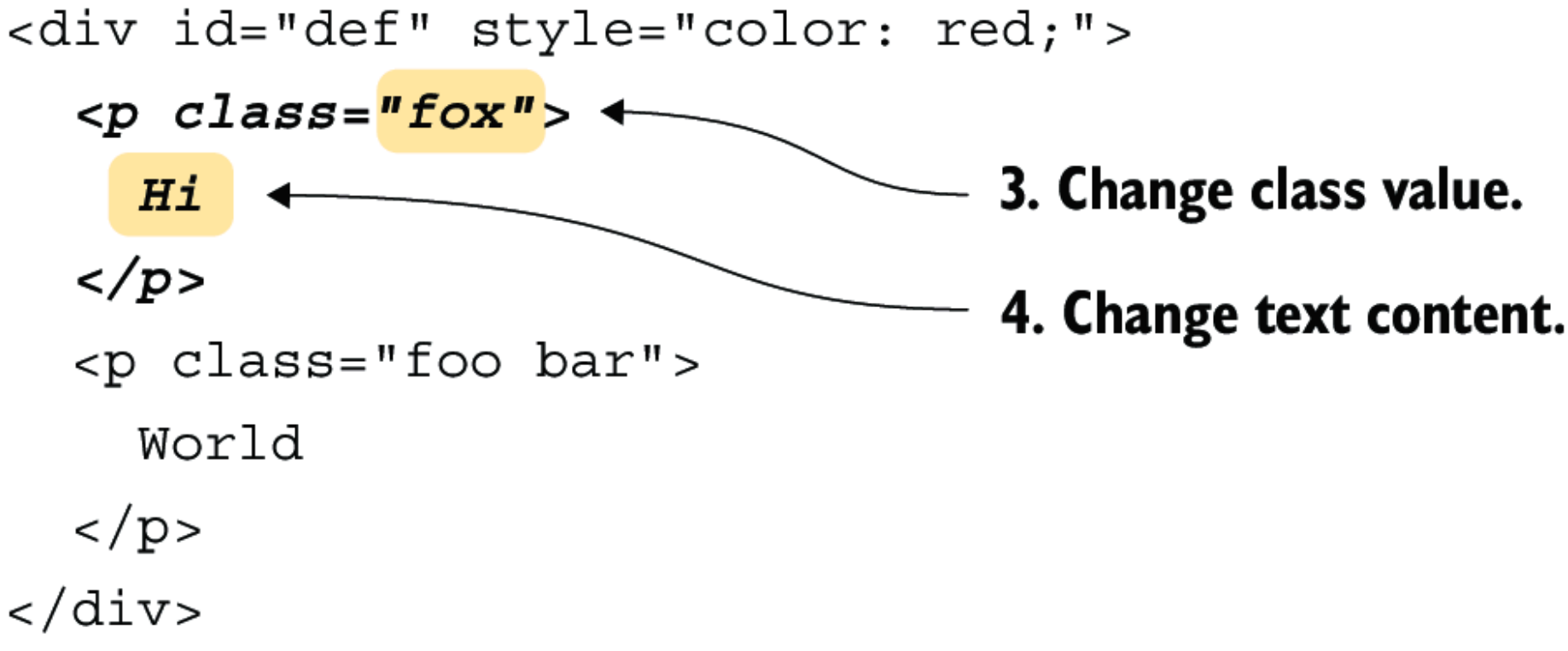

1. Change ID value.
2. Change color value.

Hello

World

MODIFYING THE CLASS ATTRIBUTE AND TEXT CONTENT OF THE FIRST <P> NODE

```
p1.className = 'fox'
p1Text.nodeValue = 'Hi'
```


<div id="def" style="color: red;">
<p class="fox">
Hi
</p>
<p class="foo bar">
World
</p>
</div>
3. Change class value.
4. Change text content.

ADDING THE NEW <SPAN> NODE

```
mountDOM(spanVdom, div)
```

ADDING THE NEW <SPAN> NODE

```
mountDOM(spanVdom, div)
```

- This will add the node last, and not in the second place

ADDING THE NEW <SPAN> NODE

```
mountDOM(spanVdom, div)
```

- This will add the node last, and not in the second place
- We need to modify the `moundDOM()` function

ADDING THE NEW <SPAN> NODE

```
mountDOM(spanVdom, div)
```

- This will add the node last, and not in the second place
- We need to modify the `moundDOM( )` function

```
mountDOM(spanVdom, div, 1)
```

```
<div id="def" style="color: red;">
  <p class="fox">
    Hi
  </p>
  <span class="foo">
    there
  </span>
  <p class="foo bar">
    World
  </p>
</div>
```

5. Add element node.

MODIFYING THE CLASS ATTRIBUTE AND TEXT CONTENT OF THE SECOND <P> NODE

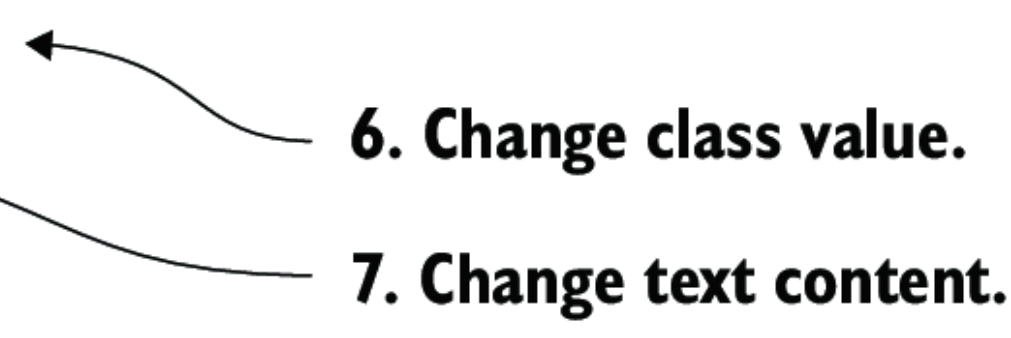
- The node after the insertion of the `<span>` will be in third place
- Just change the `class` attribute

```
p2.classList.remove('foo')
p2.classList.add('baz')
```

- The text content changed from `"World"` to `"World!"`:

```
p2Text.nodeValue = 'World!'
```

```
<div id="def" style="color: red;">
  <p class="fox">
    Hi
  </p>
  <span class="foo">
    there
  </span>
  <p class="baz bar">
    World!
  </p>
</div>
```



- This should be the output of the `patchDOM()` function

CHANGES IN THE RENDERING

CHANGES IN THE RENDERING

- The renderer to be split into three sections:

CHANGES IN THE RENDERING

- The renderer to be split into three sections:
  - mounting,

CHANGES IN THE RENDERING

- The renderer to be split into three sections:
  - mounting,
  - patching,

CHANGES IN THE RENDERING

- The renderer to be split into three sections:
  - mounting,
  - patching,
  - unmounting.

CHANGES IN THE RENDERING

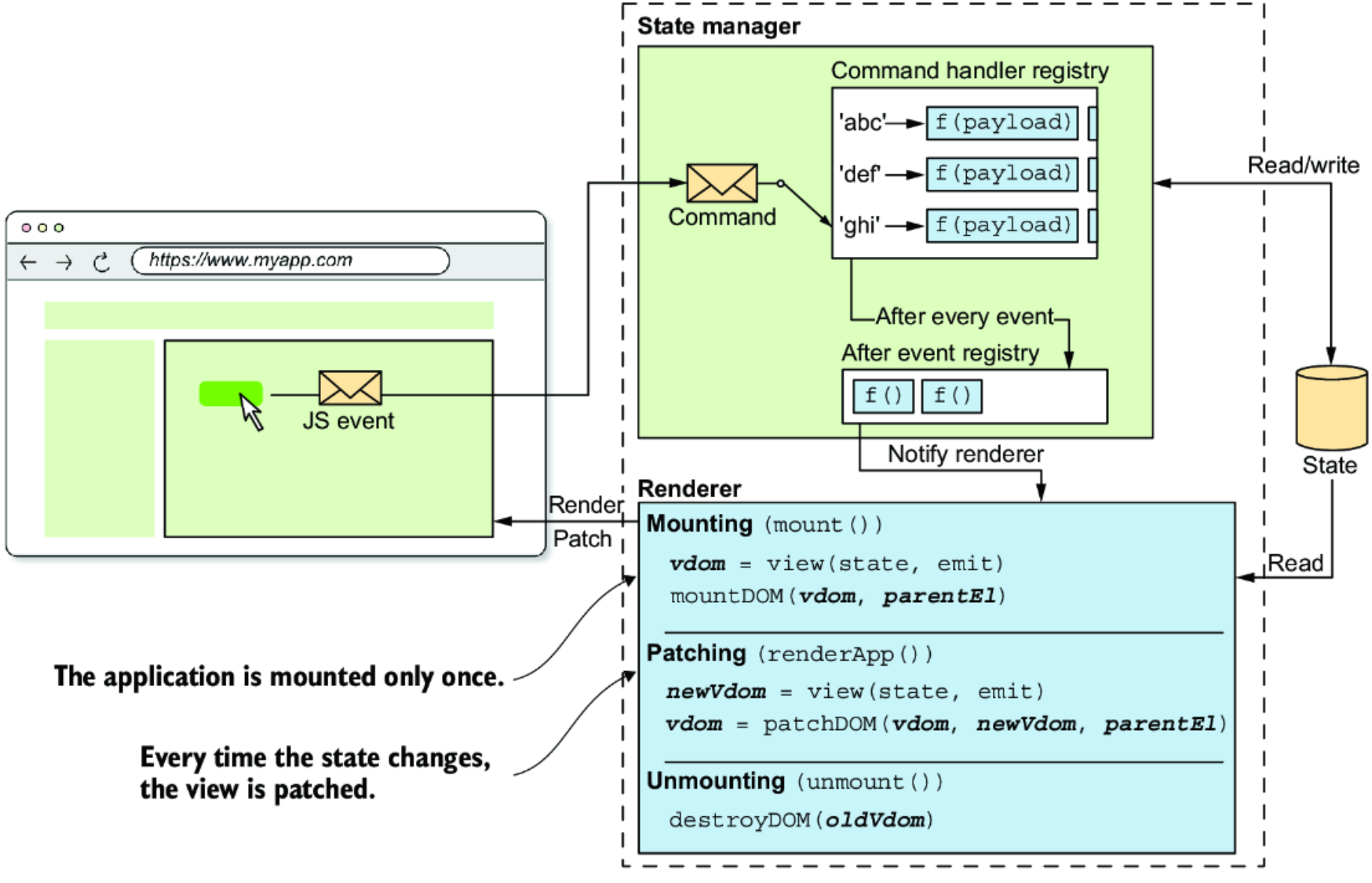
- The renderer to be split into three sections:
  - mounting,
  - patching,
  - unmounting.
- Split the `render()` function

CHANGES IN THE RENDERING

- The renderer to be split into three sections:
  - mounting,
  - patching,
  - unmounting.
- Split the `render()` function
  - `mount()`—Internally uses `mountDOM()`

CHANGES IN THE RENDERING

- The renderer to be split into three sections:
  - mounting,
  - patching,
  - unmounting.
- Split the `render()` function
  - `mount()`—Internally uses `mountDOM()`
  - `renderApp()`—Internally uses `patchDOM()`


State manager
Command handler registry
'abc'
f(payload)
'def'
f(payload)
'ghi'
f(payload)
Command
Read/write
After every event
After event registry
f()
f()
Notify renderer
State
https://www.myapp.com
JS event
Render
Patch
Renderer
Mounting (mount())
vdom = view(state, emit)
mountDOM(vdom, parentEl)
Read
Patching (renderApp())
newVdom = view(state, emit)
vdom = patchDOM(vdom, newVdom, parentEl)
Unmounting (unmount())
destroyDOM(oldVdom)
The application is mounted only once.
Every time the state changes, the view is patched.

- The `patchDOM()` takes:

- The `patchDOM()` takes:
  - the last saved virtual DOM (stored in the `vdom`)

- The `patchDOM()` takes:
  - the last saved virtual DOM (stored in the `vdom`)
  - the new virtual DOM resulting from calling the `view()` function,

- The `patchDOM()` takes:
  - the last saved virtual DOM (stored in the `vdom`)
  - the new virtual DOM resulting from calling the `view()` function,
  - the parent element (stored in the `parentEl` instance) of the DOM,

CHANGES IN THE RENDERING

CHANGES IN THE RENDERING

- The `mount()` method doesn't need to use the `renderApp()` function anymore;

CHANGES IN THE RENDERING

- The `mount()` method doesn't need to use the `renderApp()` function anymore;
- `renderApp()` is called only when the state changes.

CHANGES IN THE RENDERING

- The `mount()` method doesn't need to use the `renderApp()` function anymore;
- `renderApp()` is called only when the state changes.
- `mount()` calls the `view()` function to get the virtual DOM and then calls the `mountDOM()` function to mount the DOM.

CHANGES IN THE RENDERING

- The `mount()` method doesn't need to use the `renderApp()` function anymore;
- `renderApp()` is called only when the state changes.
- `mount()` calls the `view()` function to get the virtual DOM and then calls the `mountDOM()` function to mount the DOM.
- The `mount()` method should be called only once.

```
import { destroyDOM } from './destroy-dom'
import { Dispatcher } from './dispatcher'
import { mountDOM } from './mount-dom'
import { patchDOM } from './patch-dom'

export function createApp({ state, view, reducers = {} }) {
  let parentEl = null
  let vdom = null

 // --snip-- //
```

```
function renderApp() {
  if (vdom) {
    destroyDOM(vdom)
  }
  // Computes the new virtual DOM
  const newVdom = view(state, emit)
  mountDOM(vdom, parentEl)
  // Patches the DOM every time the state changes
  vdom = patchDOM(vdom, newVdom, parentEl)
}
```

```
  return {
    mount(_parentEl) {
      parentEl = _parentEl
      renderApp()
      vdom = view(state, emit)
      // Mount the DOM only once
      mountDOM(vdom, parentEl)
    },
 // --snip-- //
  }
}
```

DISALLOW MOUNTING MORE THAN ONCE

```
export function createApp({ state, view, reducers = {} }) {
  let parentEl = null
  let vdom = null
  let isMounted = false // Added

  // -- snip -- //
```

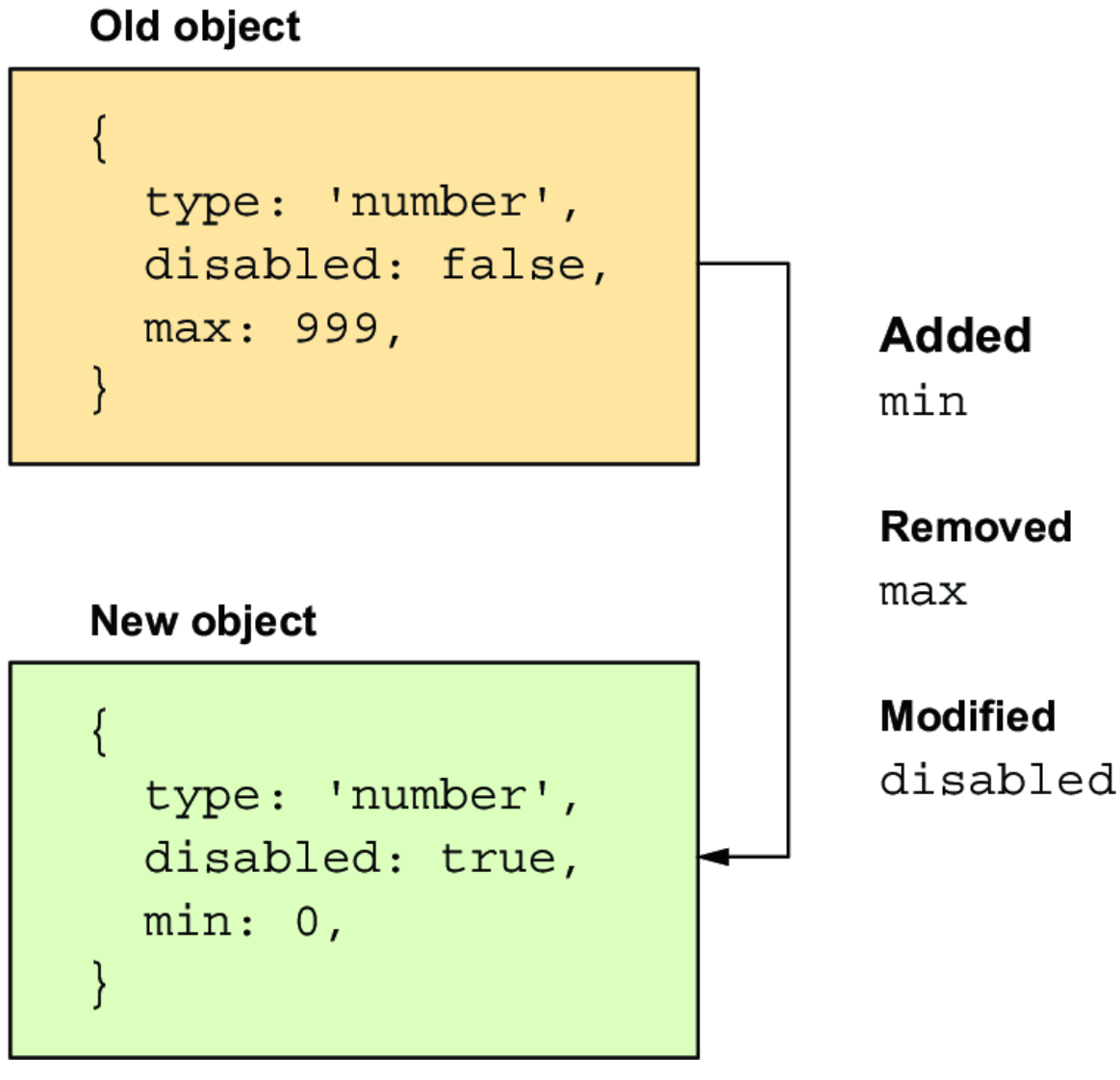
```
return {
  mount(_parentEl) {
    // Check if mounted
    if (isMounted) {
      throw new Error('The application is already mounted')
    }

    parentEl = _parentEl
    vdom = view(state, emit)
    mountDOM(vdom, parentEl)
    // Set the isMounted
    isMounted = true
  },
```

```
    unmount() {
      destroyDOM(vdom)
      vdom = null
      subscriptions.forEach((unsubscribe) => unsubscribe())
      // Set isMounted as
      isMounted = false
    },
  }
}
```

DIFFING OBJECTS

- Find the differences between the attributes of the two nodes.
- This process is called **diffing**.

DIFFING OBJECTS



THE PROCEDURE

THE PROCEDURE

1. Take a key in the old object. If you don’t see it in the new object, you know that the key was removed. Repeat with all keys.

THE PROCEDURE

1. Take a key in the old object. If you don’t see it in the new object, you know that the key was removed. Repeat with all keys.
2. Take a key in the new object. If you don’t see it in the old object, you know that the key was added. Repeat with all keys.

THE PROCEDURE

1. Take a key in the old object. If you don’t see it in the new object, you know that the key was removed. Repeat with all keys.
2. Take a key in the new object. If you don’t see it in the old object, you know that the key was added. Repeat with all keys.
3. Take a key in the new object. If you see it in the old object and the value associated with the key is different, you know that the value associated with the key changed.

Create `utils/objects.js`, and write the `objectsDiff()` function

```
// FILE: utils/objects.js
export function objectsDiff(oldObj, newObj) {
  const oldKeys = Object.keys(oldObj)
  const newKeys = Object.keys(newObj)
```

```
  return {
    // Keys in the new object that are not in the old object were a
    added: newKeys.filter((key) => !(key in oldObj)),

    //  Keys in the old object that are not in
    // the new object were removed.
    removed: oldKeys.filter((key) => !(key in newObj)),

    // Keys in both objects that have different
    // values were changed
    updated: newKeys.filter(
      (key) => key in oldObj && oldObj[key] !== newObj[key]
    ),
  }
}
```

DIFFING ARRAYS

DIFFING ARRAYS

- Useful for diffing array based props (e.g. `class`)

DIFFING ARRAYS

- Useful for diffing array based props (e.g. `class`)

```
h('p', { class: ['foo', 'bar'] }, ['Hello world'])
```

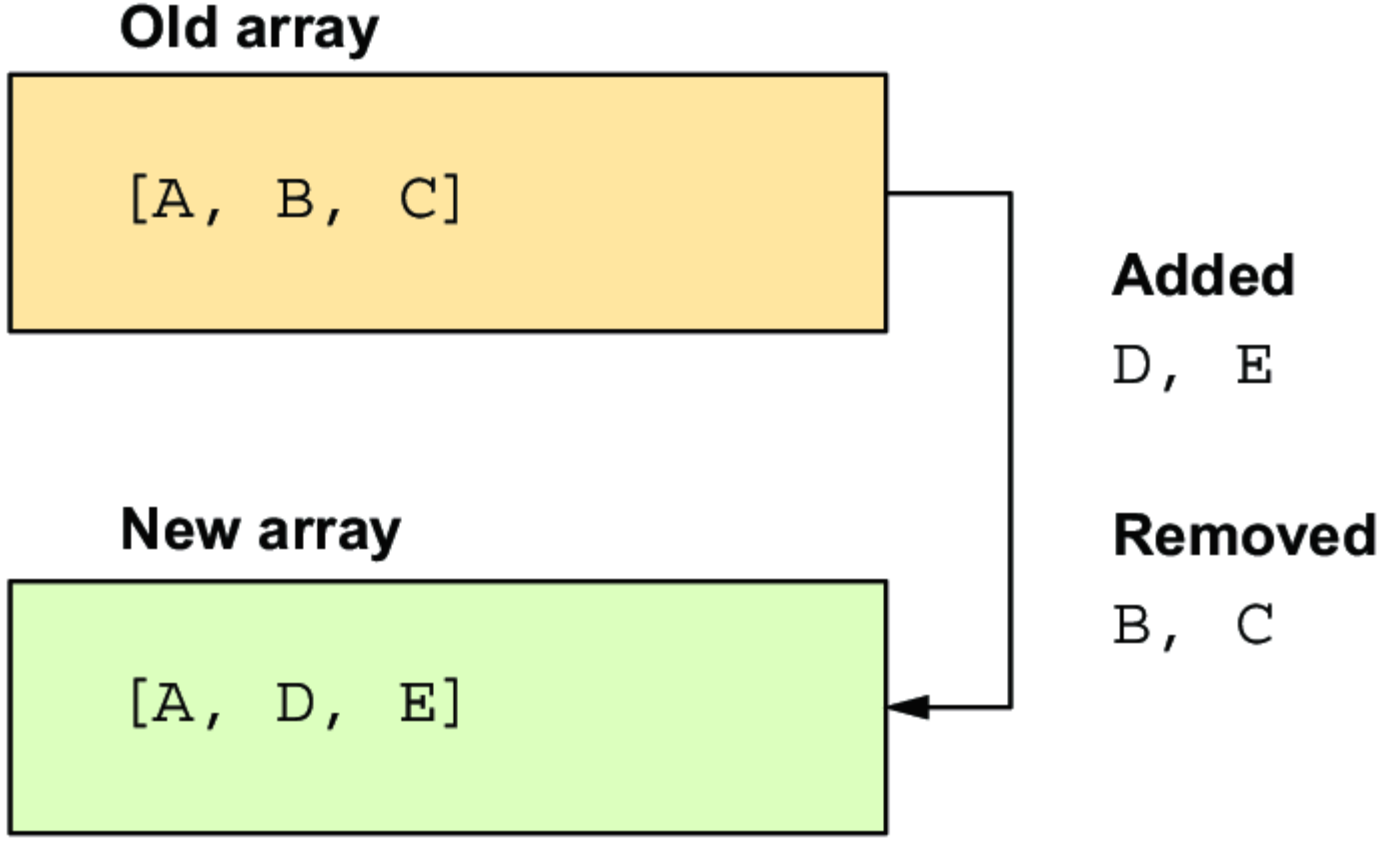
DIFFING ARRAYS

- Useful for diffing array based props (e.g. `class`)

```
h('p', { class: ['foo', 'bar'] }, ['Hello world'])
```

- The order in this example (`class`) is not relevant

- implement the `arraysDiff()` function



- A appears in both arrays.
- B and C were removed (appear only in the old array).
- D and E were added (appear only in the new array).

- in this comparison case, items are added or removed

- in this comparison case, items are added or removed
- If an item is changed:

- in this comparison case, items are added or removed
- If an item is changed:
    - remove the old item

- in this comparison case, items are added or removed
- If an item is changed:
    - remove the old item
    - add the new item.

```
// FILE: utils/arrays.js
export function arraysDiff(oldArray, newArray) {
  return {
    added: newArray.filter(
      //  Items in the new array that are
      //  not in the old array were added.
      (newItem) => !oldArray.includes(newItem)
    ),
    removed: oldArray.filter(
      // Items in the old array that are
      // not in the new array were removed
      (oldItem) => !newArray.includes(oldItem)
    ),
  }
}
```

- In the `arraysDiff()` function, you simply add or remove the classes.
- This approach might be problematic
  - the order of classes in `classList` matters.
  - Maintain the order of classes in the `classList`.
- TODO: improve `arraysDiff()` to preserve the order

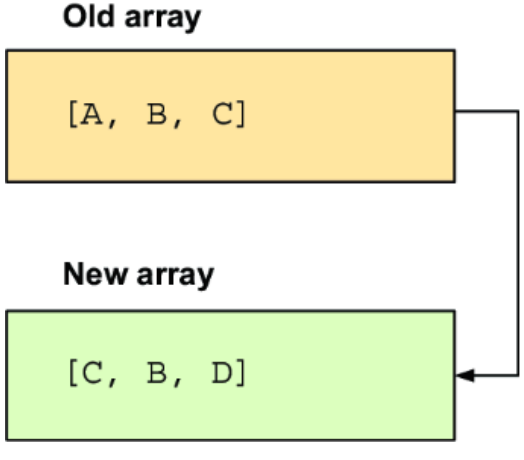
DIFFING ARRAYS AS A SEQUENCE OF OPERATIONS

DIFFING ARRAYS AS A SEQUENCE OF OPERATIONS

- For two arrays, come up with a list of `add`, `remove`, and `move` operations that transform the first array into the second one.

DIFFING ARRAYS AS A SEQUENCE OF OPERATIONS

- For two arrays, come up with a list of add, `remove`, and `move` operations that transform the first array into the second one.



Operations

1. Remove A from `i=0`. (result: ***[B, C]***)
2. Move C (originally at 2) from `i=1` to `i=0`. (result: ***[C, B]***)
3. Add D at `i=2`. (result: ***[C, B, D]***)

- The indices in the operations are always relative at the moment of the operation.

- The indices in the operations are always relative at the moment of the operation.
- **But** keep track of the original index of the item.

- The indices in the operations are always relative at the moment of the operation.
- **But** keep track of the original index of the item.
- The sequence of operations that transforms the old array into the new array **isn’t unique**.

- The indices in the operations are always relative at the moment of the operation.
- **But** keep track of the original index of the item.
- The sequence of operations that transforms the old array into the new array **isn’t unique**.
- Minimize the number of operations.

DEFINING THE OPERATIONS YOU CAN USE

The operations will be:

DEFINING THE OPERATIONS YOU CAN USE

The operations will be:

- add,

DEFINING THE OPERATIONS YOU CAN USE

The operations will be:

- `add`,
- `remove`,

DEFINING THE OPERATIONS YOU CAN USE

The operations will be:

- `add`,
- `remove`,
- `move`,

DEFINING THE OPERATIONS YOU CAN USE

The operations will be:

- `add`,
- `remove`,
- `move`,
- `noop` (no operation).

- noop operation when an item is in both arrays

- noop operation when an item is in both arrays
  - no action to stay where it *ends up*.

- noop operation when an item is in both arrays
  - no action to stay where it *ends up*.
  - Items will *move naturally* to their final position because of other item manipulations.

EXAMPLE

EXAMPLE

- If you want to go from `[A, B]` to `[B]`,
  - simply remove A and B falls into place naturally.

EXAMPLE

- If you want to go from `[A, B]` to `[B]`,
  - simply remove A and B falls into place naturally.
- When an item moves naturally, include a noop operation in the sequence.

add

add

```
{ op: 'add', item: 'D', index: 2}
```

add

```
  { op: 'add', item: 'D', index: 2}
```

remove

add

```
  { op: 'add', item: 'D', index: 2}
```

remove

```
  {op: 'remove', item: 'B', index:0}
```

add

```
  { op: 'add', item: 'D', index: 2}
```

remove

```
  {op: 'remove', item: 'B', index:0}
```

move

add

```
{ op: 'add', item: 'D', index: 2}
```

remove

```
{op: 'remove', item: 'B', index:0}
```

move

```
{op:'move', item: 'C',
originalIndex: 2,from:2,index:0}
```

add

```
{ op: 'add', item: 'D', index: 2}
```

remove

```
{op: 'remove', item: 'B', index:0}
```

move

```
{op:'move', item: 'C',
originalIndex: 2,from:2,index:0}
```

noop

add

```
{ op: 'add', item: 'D', index: 2}
```

remove

```
{op: 'remove', item: 'B', index:0}
```

move

```
{op:'move', item: 'C',
originalIndex: 2,from:2,index:0}
```

noop

```
{ op: 'noop', item:'A',
originalIndex:0, index: 1}
```

op

op

that indicates the type of operation.

`op`

that indicates the type of operation.

`item`

`op`

that indicates the type of operation.

`item`

is the item added, removed, or moved (naturally or forcefully).

`op`
: that indicates the type of operation.

`item`
: is the item added, removed, or moved (naturally or forcefully).

`index`

`op`

that indicates the type of operation.

`item`

is the item added, removed, or moved (naturally or forcefully).

`index`

is the index where the item **ends up**,

- for `remove` operation, it's the index from which the item was removed.

`move` and `noop`

the `originalIndex` property indicates the index where the item started.

`move`

we also keep track of the `from` property,

```
[
  {op: 'remove', index: 0, item: 'A'}
  {op: 'move', originalIndex: 2, from: 1, index: 0, item: 'C'}
  {op: 'noop', index: 1, originalIndex: 1, item: 'B'}
  {op: 'add', index: 2, item: 'D'}
]
```

THE ALGORITHM

THE ALGORITHM

THE ALGORITHM

THE ALGORITHM

THE ALGORITHM

THE ALGORITHM

- Iterate over the indices of the new array:

THE ALGORITHM

- Iterate over the indices of the new array:
  - Let `i` be the index (`0 <= i < newArray.length`).

THE ALGORITHM

- Iterate over the indices of the new array:
  - Let `i` be the index (`0 <= i < newArray.length`).
  - Let `newItem` be the item at `i` in the new array.

THE ALGORITHM

- Iterate over the indices of the new array:
  - Let `i` be the index (`0 <= i < newArray.length`).
  - Let `newItem` be the item at `i` in the new array.
  - Let `oldItem` be the item at `i` in the old array (provided that there is one).

- If `oldItem` doesn't appear in the new array:

- If `oldItem` doesn't appear in the new array:
  - Add a `remove` operation to the sequence.

- If `oldItem` doesn't appear in the new array:
  - Add a `remove` operation to the sequence.
  - Remove the `oldItem` from the array.

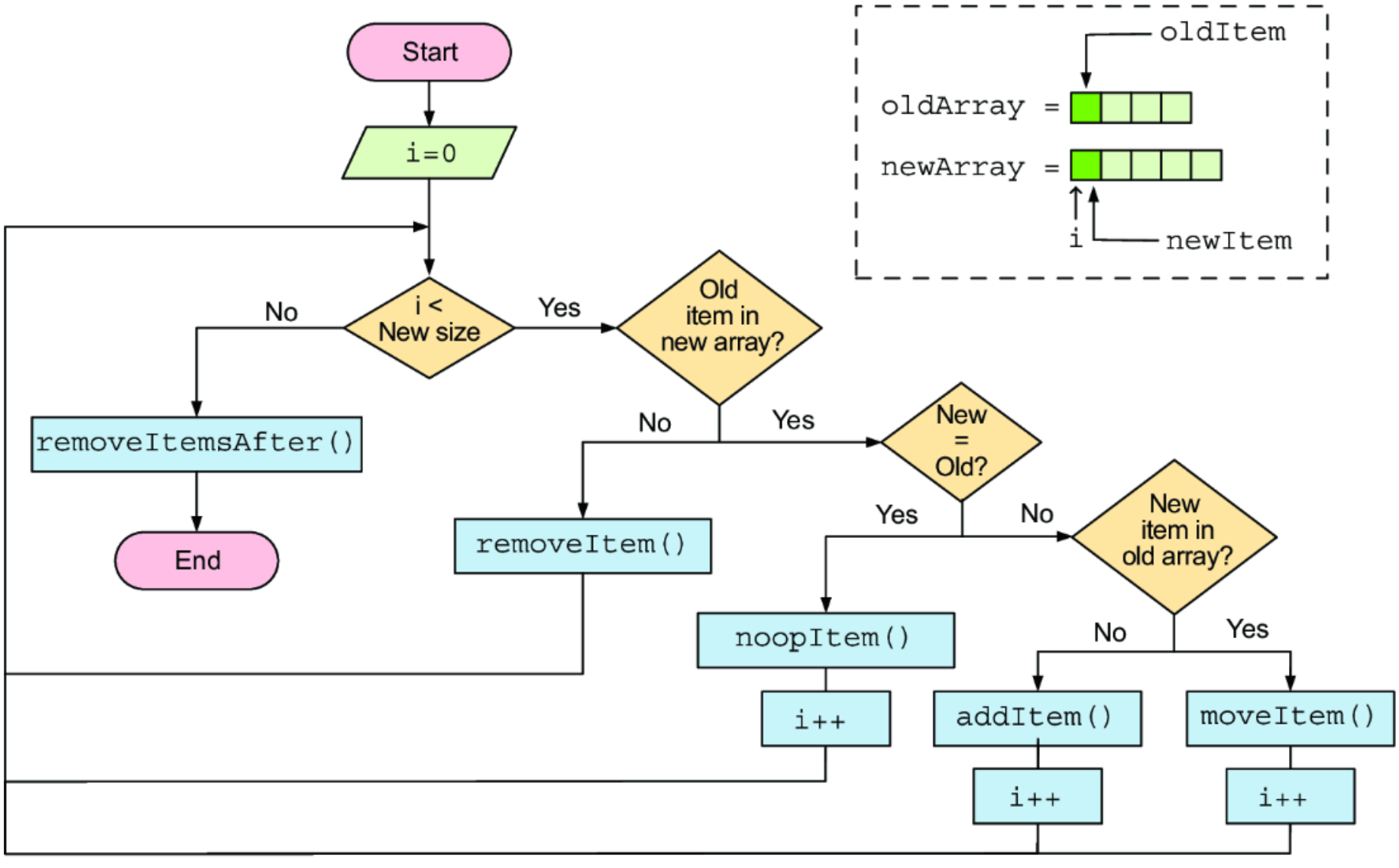
- If `oldItem` doesn't appear in the new array:
  - Add a `remove` operation to the sequence.
  - Remove the `oldItem` from the array.
  - Start again from step `1` without incrementing `i` (stay at the same index).

- If `newItem == oldItem`:
  - Add a `noop` operation to the sequence, using the `oldItem` original index (its index at the beginning of the process).
  - Start again from step `1`, incrementing `i`.

- If `newItem != oldItem` and `newItem` can't be found in the old array starting at `i`:
  - Add an `add` operation to the sequence.
  - Add the `newItem` to the old array at `i`.
  - Start again from step `1`, incrementing `i`.

- If `newItem != oldItem` and `newItem` can be found in the old array starting at `i`:
  - Add a `move` operation to the sequence, using the `oldItem` current index and the original index.
  - Move the `oldItem` to `i`.
  - Start again from step `1`, incrementing `i`.

- If `i` is greater than the length of `newArray`:
  - Add a `remove` operation for each remaining item in `oldArray`.
  - Remove all remaining items in `oldArray`.
  - Stop the algorithm.


Start
i=0
i < New size
No
Yes
removeItemsAfter()
End
Old item in new array?
No
Yes
removeItem()
New = Old?
Yes
No
noopItem()
i++
New item in old array?
No
Yes
addItem()
i++
moveItem()
i++
oldItem
oldArray =
newArray =
i
newItem

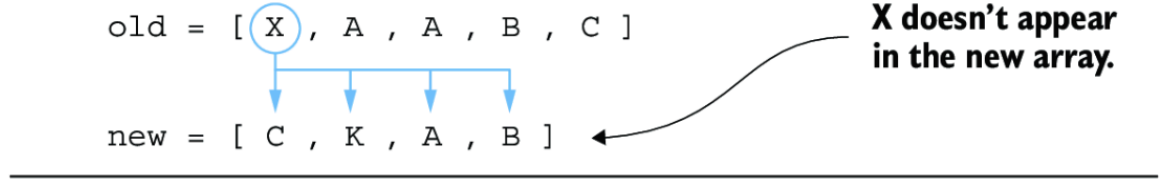
EXAMPLE BY HAND

- `oldArray = [X,A,A,B,C]`
- `newArray = [C,K,A,B]`

STEP 1 (I=0)

- Remove X

i=0

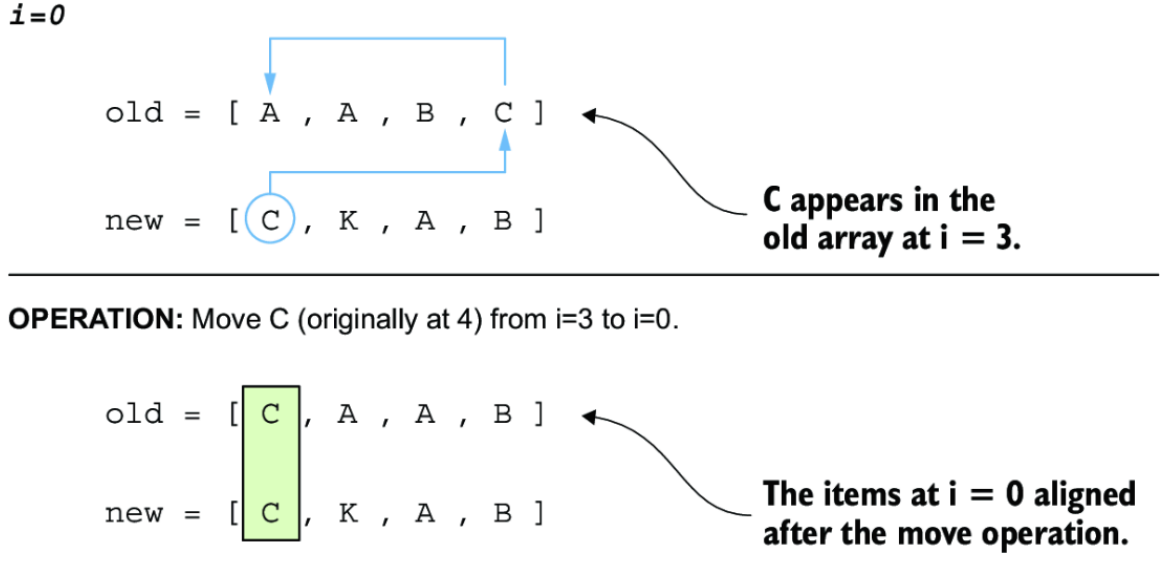


OPERATION: Remove X at i=0.

```
old = [ X , A , A , B , C ]

new = [ C , K , A , B ]
```

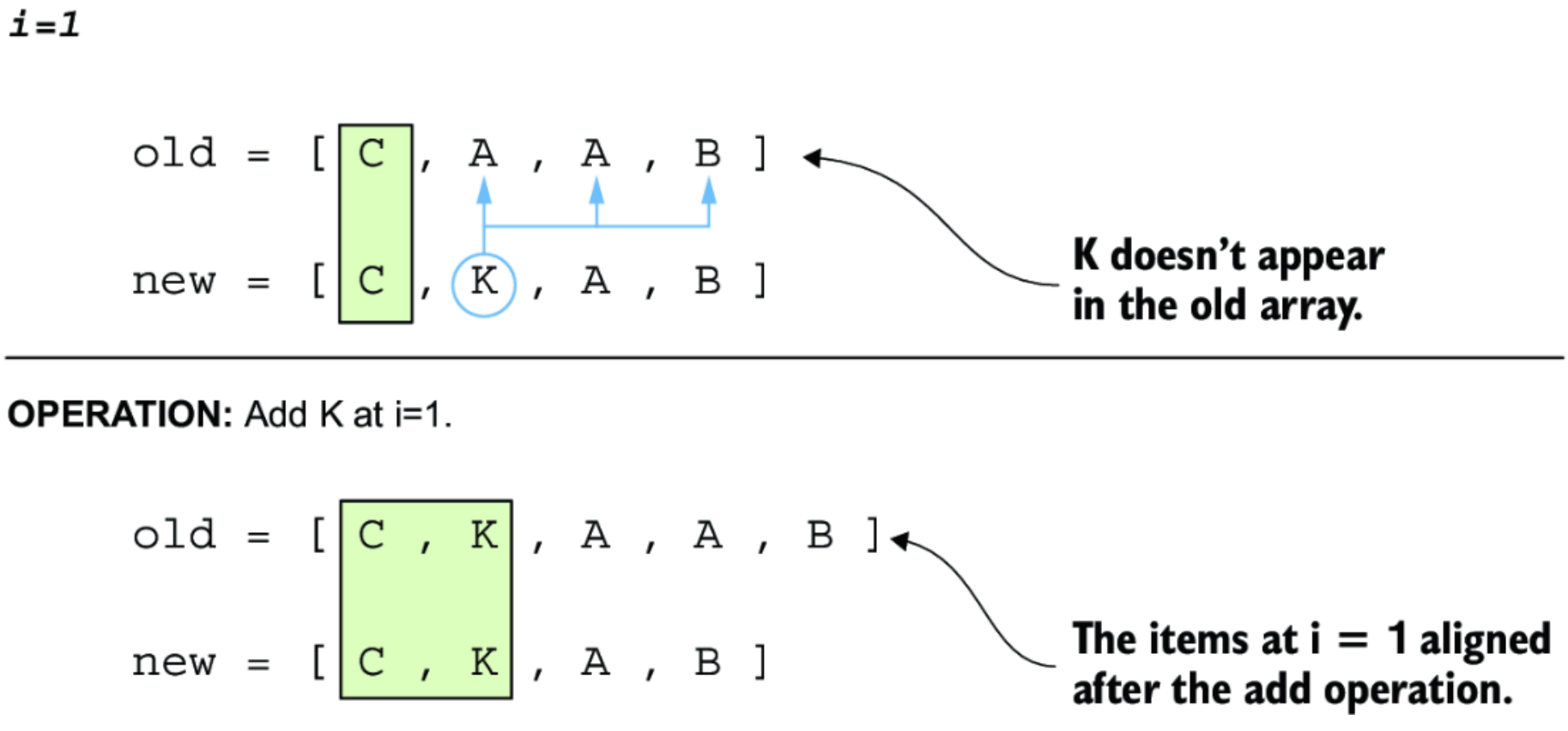
STEP 2 (I=0)

- Move C in the new array, as C appears at `i_3`



STEP 3 (I=1)

- add K in the new array.



STEP 4 (I=2)

At `i=2`, both arrays have an A but A moved naturally (noop):

STEP 4 (I=2)

At `i=2`, both arrays have an A but A moved naturally (noop):

- It moved one position to the left when the X was removed (step 1).

STEP 4 (I=2)

At `i=2`, both arrays have an A but A moved naturally (noop):

- It moved one position to the left when the X was removed (step 1).
- It moved one position to the right when the C was moved (step 2), which canceled the previous move.

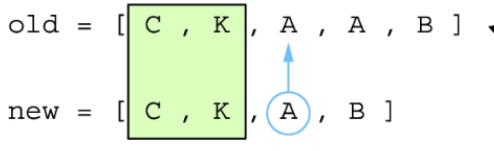
STEP 4 (I=2)

At `i=2`, both arrays have an A but A moved naturally (noop):

- It moved one position to the left when the X was removed (step 1).
- It moved one position to the right when the C was moved (step 2), which canceled the previous move.
- It moved one position to the right when the K was added (step 3).

- A moved one position to the right,
  - subtract 1 from the current index of the A in the old array, making the A's original index `i=1`.

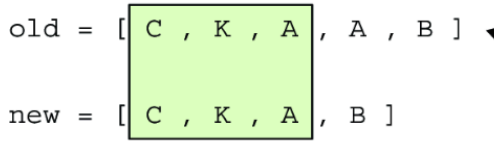
i=2



A already appears in the correct position of the old array.

OPERATION: Noop A (originally at 1) to i=2.

A was originally at i = 1, but it moved due to the previous operations.

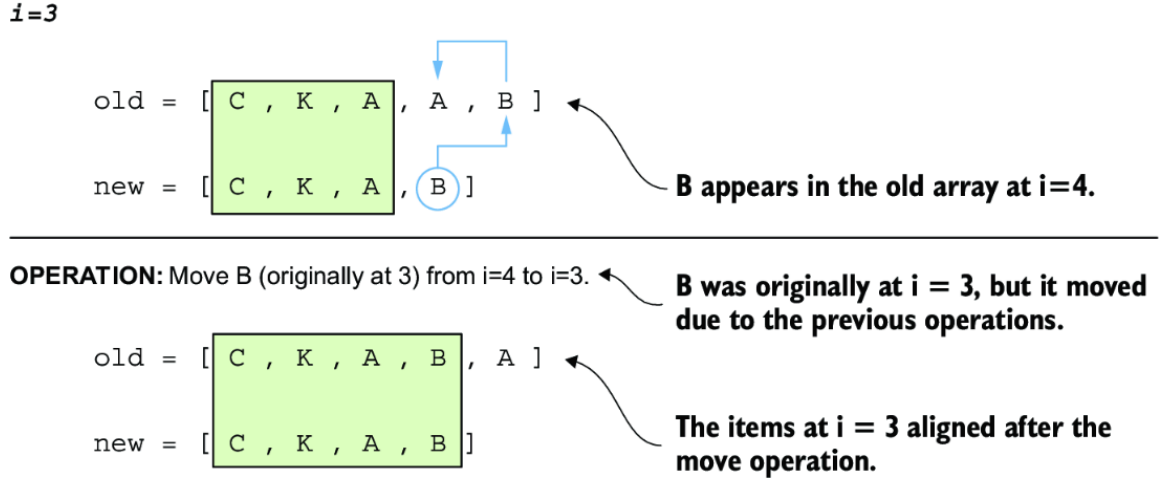


The items at i = 2 were already aligned.

- keep track of the old array items' original indexes so that you'll have them available when you add `noop` and `move` operations.

STEP 5 (I=3)

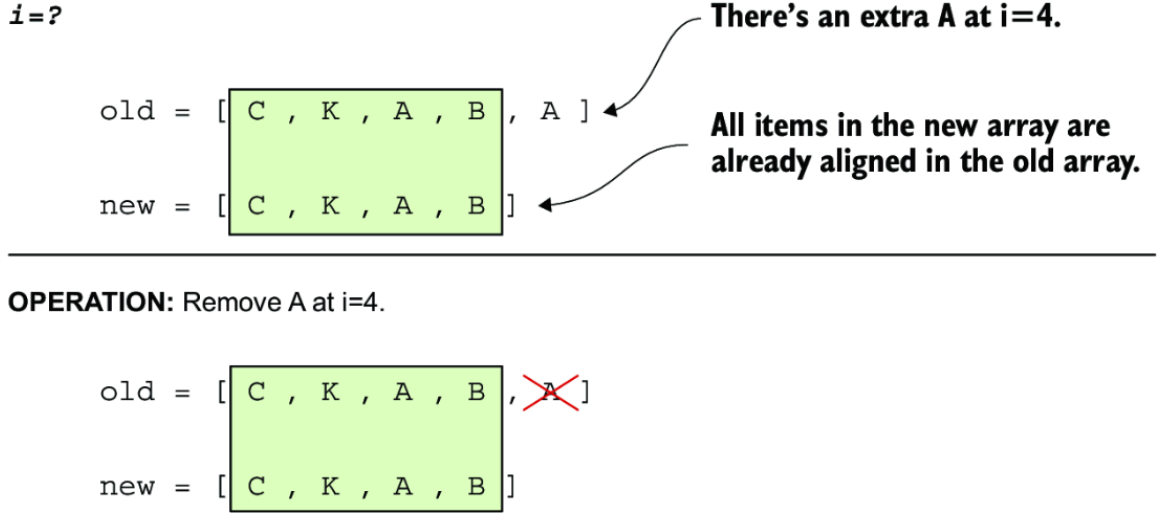
At `i=3`, B should be moved at `i=4`



- All items in the new array are processed
- remove excess items in the old array.

STEP 6

- At `i=4` in the old array the extra A is removed



IMPLEMENTING THE ALGORITHM

OPERATION NAMES CONSTANT

```
// FILE: utils/arrays.js
export const ARRAY_DIFF_OP = {
  ADD: 'add',
  REMOVE: 'remove',
  MOVE: 'move',
  NOOP: 'noop',
}
```

TRACKING ORIGINAL ARRAY

- To keep track of the old array’s original indices

- To keep track of the old array's original indices
  - create a `ArrayWithOriginalIndices` class

- To keep track of the old array's original indices
  - create a `ArrayWithOriginalIndices` class
  - wraps a copy of the old array and keeps track of the original indices.

- To keep track of the old array's original indices
  - create a `ArrayWithOriginalIndices` class
  - wraps a copy of the old array and keeps track of the original indices.
  - This wrapper class will search for a given item in the wrapped array

- To keep track of the old array's original indices
  - create a `ArrayWithOriginalIndices` class
  - wraps a copy of the old array and keeps track of the original indices.
  - This wrapper class will search for a given item in the wrapped array
  - The items in the array will be virtual nodes

- Can not rely on the default === comparison

- Can not rely on the default === comparison
- A custom comparison function needs to be implemented

- Can not rely on the default === comparison
- A custom comparison function needs to be implemented

```
const a = { foo: 'bar' }
const b = { foo: 'bar' }
console.log(a === b) // false
```

IMPLEMENTATION

```
// FILE: utils/arrays.js
class ArrayWithOriginalIndices {
  #array = []
  #originalIndices = []
  #equalsFn

  constructor(array, equalsFn) {
    // The internal array is a copy of the original.
    this.#array = [...array]
    // Keeps track of the original indices
    this.#originalIndices = array.map((_, i) => i)
    // Saves the function used to compare items in the array
    this.#equalsFn = equalsFn                          #4
  }
```

```
  get length() {
    // The current length of the array
    return this.#array.length
  }
}
```

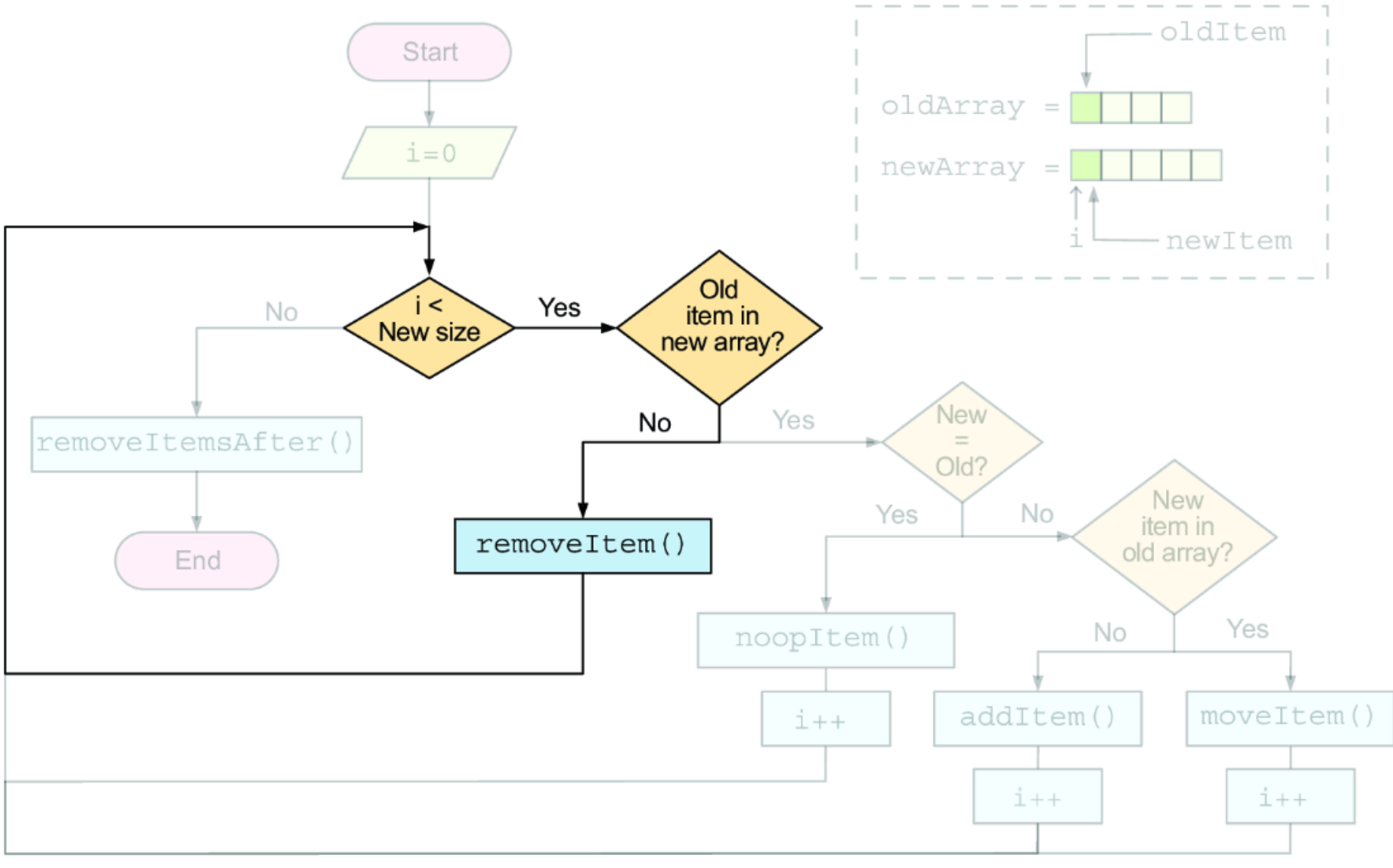
arraysDiffSequence() FUNCTION

```
//FILE: utils/arrays.js
export function arraysDiffSequence(
  oldArray,
  newArray,
  // The equalsFn is used to compare items in the array (=== for now
  equalsFn = (a, b) => a === b
 )
 {
  const sequence = []
  // Wraps the old array in an ArrayWithOriginalIndices instance
  const array = new ArrayWithOriginalIndices(oldArray, equalsFn)
```

```
  // Iterates the indices of the new array
  for (let index = 0; index < newArray.length; index++) {
    // TODO: removal case

    // TODO: noop case

    // TODO: addition case

    // TODO: move case
 }

  // TODO: remove extra items

  // Returns the sequence of operations
  return sequence       #4
}
```

THE REMOVE CASE

- If item was removed
  - Check whether the item in the old array at the current index doesn’t exist in the new array.


Start
i=0
i <
New size
No
Yes
Old
item in
new array?
No
Yes
New
=
Old?
Yes
No
New
item in
old array?
No
Yes
removeItemsAfter()
End
removeItem()
noopItem()
addItem()
moveItem()
i++
i++
i++
oldItem
oldArray =
newArray =
i
newItem

- `isRemoval()` method in `ArrayWithOriginalIndices`

```
//FILE: utils/arrays.js
class ArrayWithOriginalIndices {
  #array = []
  #originalIndices = []
  #equalsFn

  constructor(array, equalsFn) {
    this.#array = [...array]
    this.#originalIndices = array.map((_, i) => i)
    this.#equalsFn = equalsFn
  }

  get length() {
    return this.#array.length
  }
```

```
  isRemoval(index, newArray) {
    //  If the index is out of bounds, there's nothing to remove.
    if (index >= this.length) {
      return false
    }

    //  Gets the item in the old array at the given index
    const item = this.#array[index]

    // Tries to find the same item in the new array, returning its i
    const indexInNewArray = newArray.findIndex((newItem) =>
      // Uses the #equalsFn to compare the items
      this.#equalsFn(item, newItem)
    )

    // If the index is -1, the item was removed.
    return indexInNewArray === -1
  }
}
```

- method to handle the removal of an item and return the operation.
- reflect the removal operation in `ArrayWithOriginalIndices` instance:
  - `#array`
  - `#originalIndices`
- Add `removeItem()` method to `ArrayWithOriginalIndices`

```
//FILE: (utils/arrays.js)
class ArrayWithOriginalIndices {
  // --snip-- //
  isRemoval(index, newArray) {
  // --snip-- //
  }
  removeItem(index) {
    // Creates the operation for the removal
    const operation = {
      op: ARRAY_DIFF_OP.REMOVE,
      index,
      // The current index of the item in
      // the old array (not the original index)
      item: this.#array[index],
    }
```

```
    // Removes the item from the array
    this.#array.splice(index, 1)
    // Removes the original index of the item
    this.#originalIndices.splice(index, 1)

    // Returns the operation
    return operation
  }
}
```

- The `arraysDiffSequence()` function.

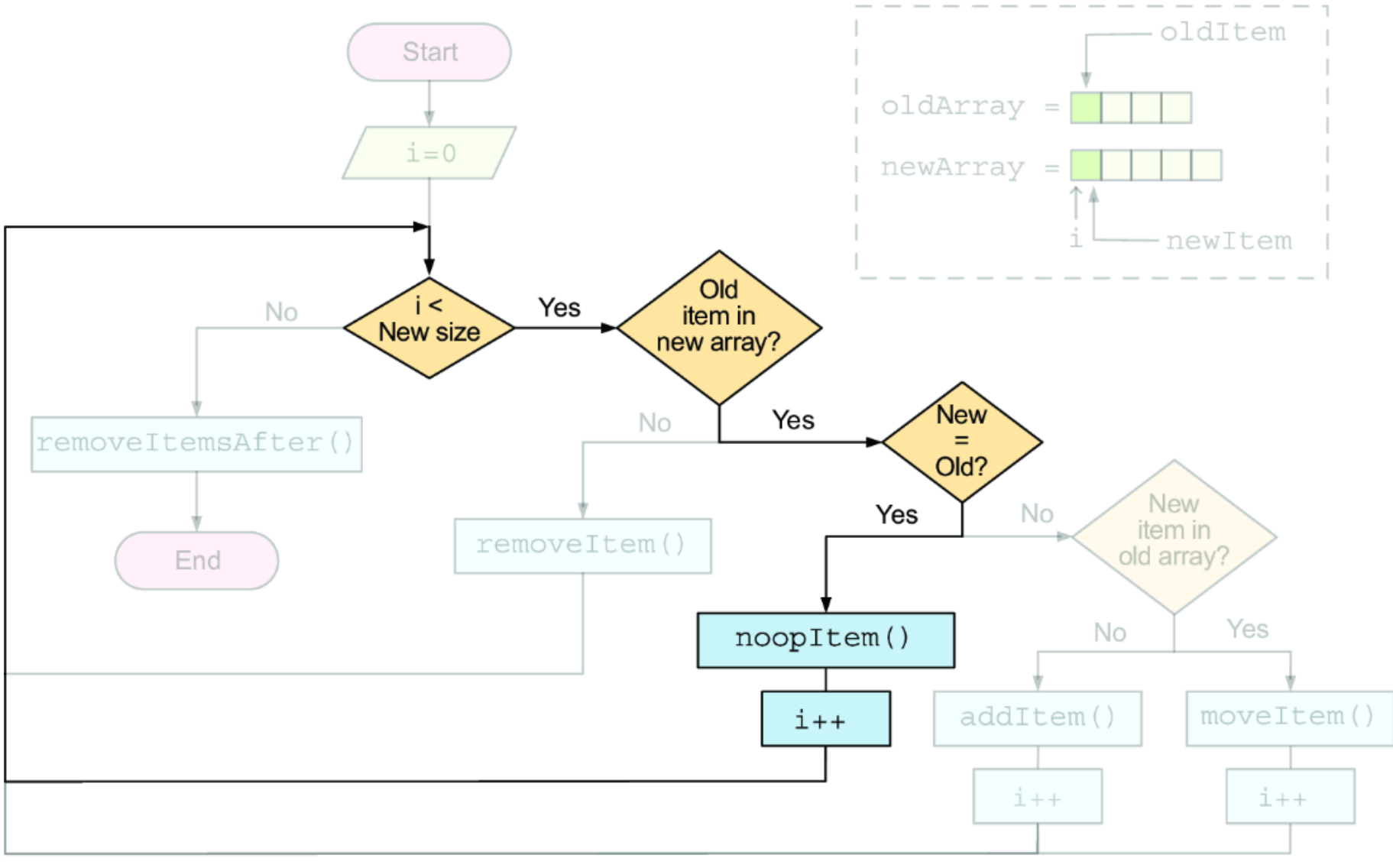
```
export function arraysDiffSequence(
  oldArray,
  newArray,
  equalsFn = (a, b) => a === b
) {
  const sequence = []
  const array = new ArrayWithOriginalIndices(oldArray, equalsFn)
```

```
for (let index = 0; index < newArray.length; index++) {
  //  Checks whether the item in the old array
  // at the current index was removed
  if (array.isRemoval(index, newArray)) {
    // Removes the item and pushes the operation to the sequence
    sequence.push(array.removeItem(index))
    // Decrements the index to stay at the
    // same index in the next iteration
    index--
    continue
  }
```

```
 // TODO: noop ecase

 // TODO: addition case

 // TODO: move case
 }

 // TODO: remove extra items

  return sequence
}
```

THE NOOP CASE

The noop case happens when, at the current index, both the old and new arrays have the same item.


Start
i=0
i <
New size
No
Yes
Old
item in
new array?
No
Yes
New
=
Old?
Yes
No
New
item in
old array?
No
Yes
removeItemsAfter()
End
removeItem()
noopItem()
i++
addItem()
i++
moveItem()
i++
oldItem
oldArray =
newArray =
i
newItem

- `isNoop()` method in the `ArrayWithOriginalIndices` class

```
// FILE: utils/arrays.js
class ArrayWithOriginalIndices {
 // --snip-- //

  isNoop(index, newArray) {
    // If the index is out of bounds, there can't be a noop.\\
    if (index >= this.length) {
      return false
    }

    // The item in the old array\\
    const item = this.#array[index]
    // The item in the new array\\
    const newItem = newArray[index]

    // Checks whether the items are equal\\
    return this.#equalsFn(item, newItem)
  }
}
```

- `noopItem()` method in `ArrayWithOriginalIndices`
- implement an `originalIndexAt()` method to get the original index of an

item in the old array.

```
class ArrayWithOriginalIndices {
 // --snip-- //

  isNoop(index, newArray) {
 // --snip-- //
  }

  originalIndexAt(index) {
  // Returns the original index of the item in the old array
     return this.#originalIndices[index]
  }
```

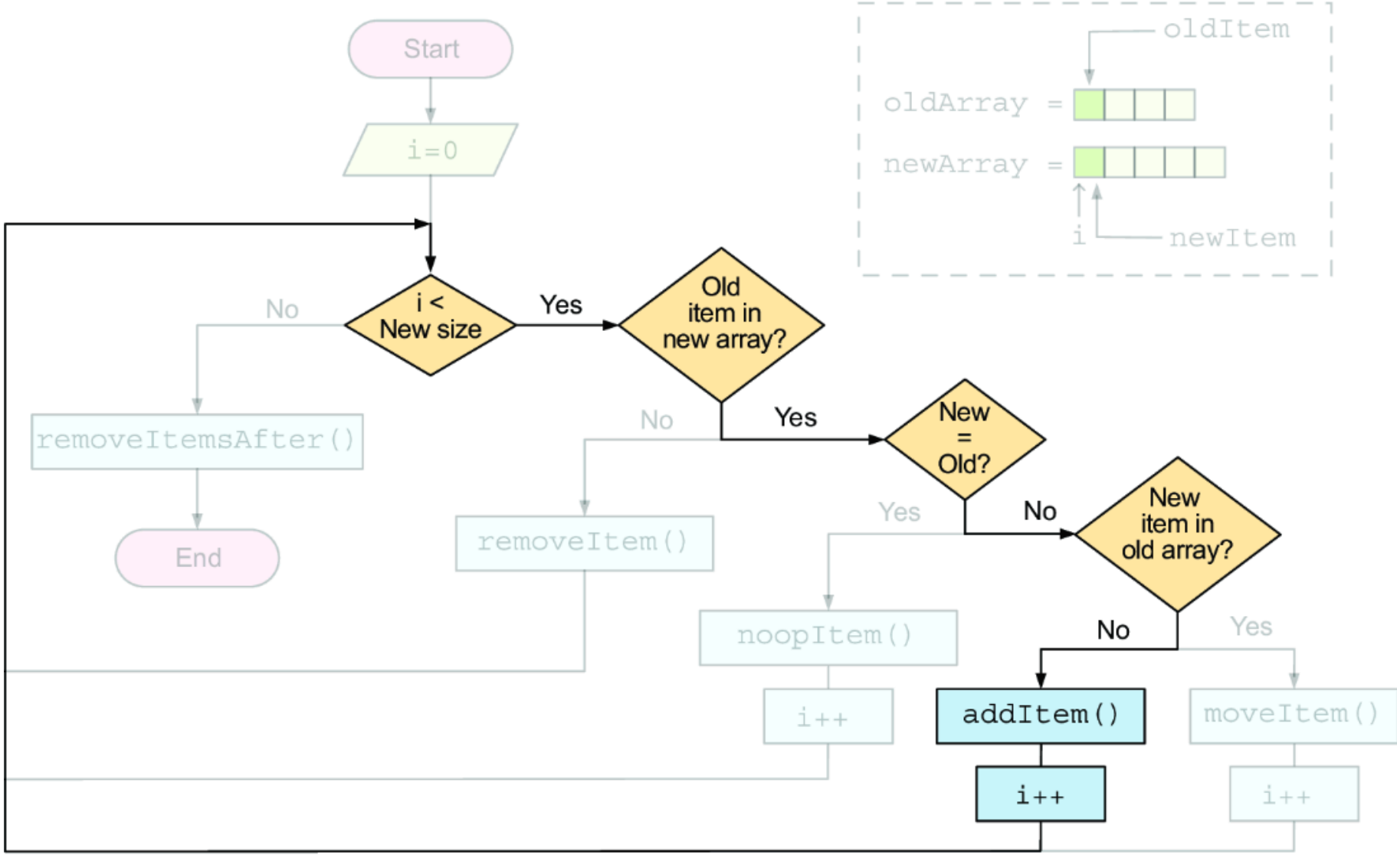
```
  noopItem(index) {
  // Creates the noop operation
   return {
      op: ARRAY_DIFF_OP.NOOP,
      // Adds the original index to the operation
      originalIndex: this.originalIndexAt(index),
      index,
      // Includes the item in the operation
      item: this.#array[index],
    }
  }
}
```

- Implement the `noop` case in the `arraysDiffSequence()` function.

```
export function arraysDiffSequence(
  oldArray,
  newArray,
  equalsFn = (a, b) => a === b
) {
  const sequence = []
  const array = new ArrayWithOriginalIndices(oldArray, equalsFn)

  for (let index = 0; index < newArray.length; index++) {
    if (array.isRemoval(index, newArray)) {
      sequence.push(array.removeItem(index))
      index--
      continue
    }
```

```
    // Checks whether the operation is a noop
    if (array.isNoop(index, newArray)) {
      // Pushes the noop operation to the sequence\\
      sequence.push(array.noopItem(index))
      continue
    }
 // TODO: addition case
 // TODO: move case
 }
 // TODO: remove extra items
  return sequence
}
```

THE ADDITION CASE



- implement `findIndexFrom()` method.
- implement `isAddition()` method

```
class ArrayWithOriginalIndices {
 // --snip-- //

// Checks whether the item exists starting at the given index\\
  isAddition(item, fromIdx) {
    return this.findIndexFrom(item, fromIdx) === -1
  }
```

```
  findIndexFrom(item, fromIndex) {
    // Starts looking from the given index\\
    for (let i = fromIndex; i < this.length; i++) {
      // If the item at the index i equals the given one, returns i
      if (this.#equalsFn(item, this.#array[i])) {
        return i
      }
    }
    // Returns -1 if the item wasn't found\\
    return -1
  }
}
```

- Add the item to the old array and return the `add` operation.
- Add an entry to the `#originalIndices` property

```
class ArrayWithOriginalIndices {
 // --snip-- //

  isAddition(item, fromIdx) {
    return this.findIndexFrom(item, fromIdx) === -1
  }

  findIndexFrom(item, fromIndex) {
    for (let i = fromIndex; i < this.length; i++) {
      if (this.#equalsFn(item, this.#array[i])) {
        return i
      }
    }

    return -1
  }
```

```
  addItem(item, index) {

    // Creates the add operation
    const operation = {
      op: ARRAY_DIFF_OP.ADD,
      index,
      item,
    }

    // Adds the new item to the old array at the given index
    this.#array.splice(index, 0, item)
    // Adds a -1 index to the #originalIndices array at the given i
    this.#originalIndices.splice(index, 0, -1)

    // Returns the add operation
    return operation
  }
}
```

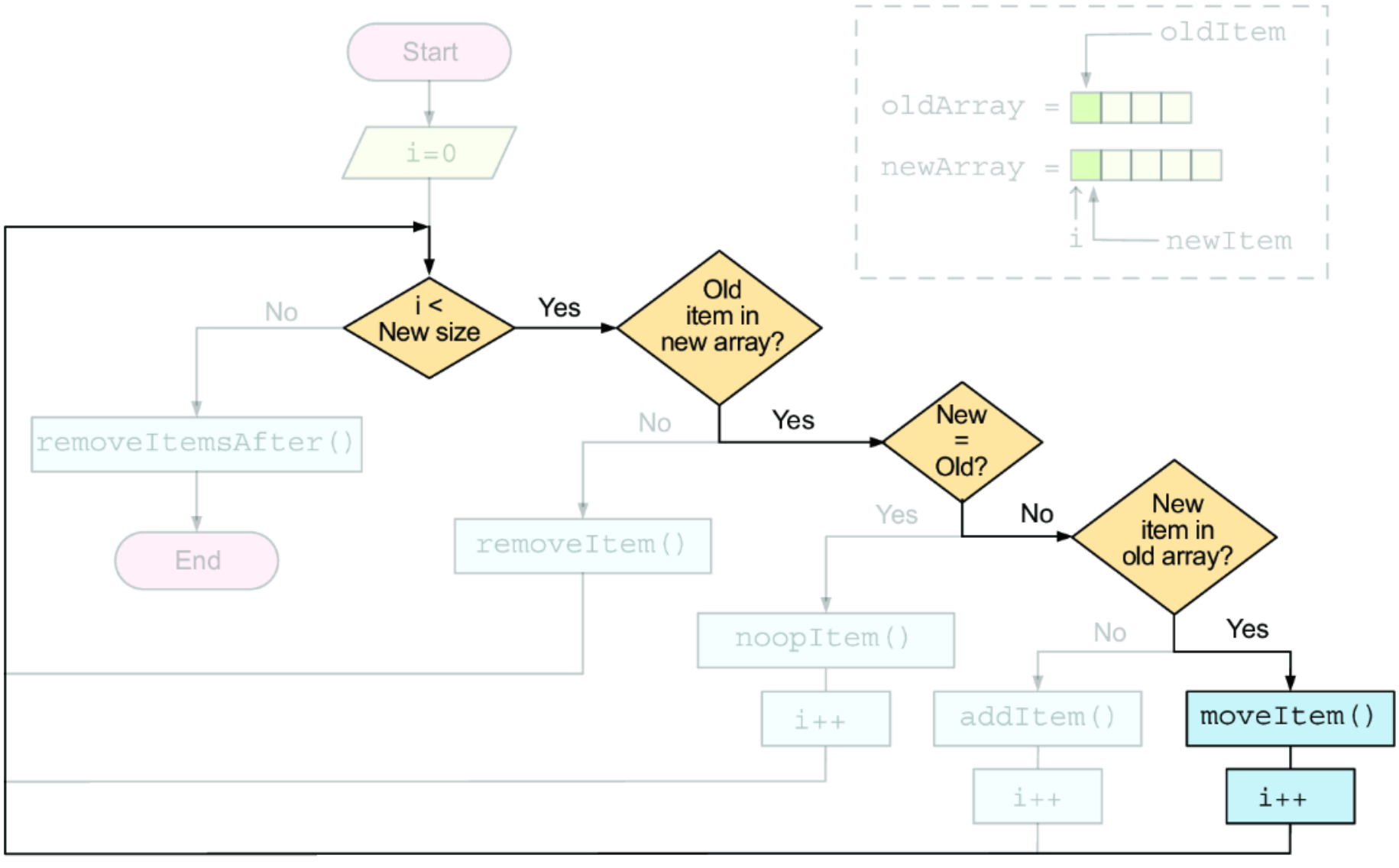
```
export function arraysDiffSequence(
  oldArray,
  newArray,
  equalsFn = (a, b) => a === b
) {
  const sequence = []
  const array = new ArrayWithOriginalIndices(oldArray, equalsFn)

  for (let index = 0; index < newArray.length; index++) {
    if (array.isRemoval(index, newArray)) {
      sequence.push(array.removeItem(index))
      index--
      continue
    }
```

```
    if (array.isNoop(index, newArray)) {
      sequence.push(array.noopItem(index))
      continue
    }

    // Gets the item in the new array at the current index\\
    const item = newArray[index]

    // Checks whether the case is an addition\\
    if (array.isAddition(item, index)) {
      // Appends the add operation to the sequence\\
      sequence.push(array.addItem(item, index))
      // Continues with the loop\\
      continue
    }
```

```
 // TODO: move case
 }

 // TODO: remove extra items
  return sequence
}
```

THE MOVE CASE

- if the operation isn't a `remove`, an `add`, or a `noop`, it must be a `move`.


oldItem
oldArray =
newArray =
i
newItem
Start
i=0
i <
New size
No
Yes
Old
item in
new array?
No
Yes
New
=
Old?
Yes
No
New
item in
old array?
No
Yes
removeItemsAfter()
End
removeItem()
noopItem()
i++
addItem()
i++
moveItem()
i++

- To move an item
  - you want to extract it from the array
  - insert it into the new position
- Two things in mind:
  - move the original index to its new position,
  - have to include the original index in the `move` operation.

```
class ArrayWithOriginalIndices {
 // --snip-- //
  moveItem(item, toIndex) {
    // Looks for the item in the old array, starting from the targe
    const fromIndex = this.findIndexFrom(item, toIndex)
```

```
    // Creates the move operation\\
    const operation = {
      op: ARRAY_DIFF_OP.MOVE,
      // Includes the original index in the operation\\
      originalIndex: this.originalIndexAt(fromIndex),
      from: fromIndex,
      index: toIndex,
      item: this.#array[fromIndex],
    }
```

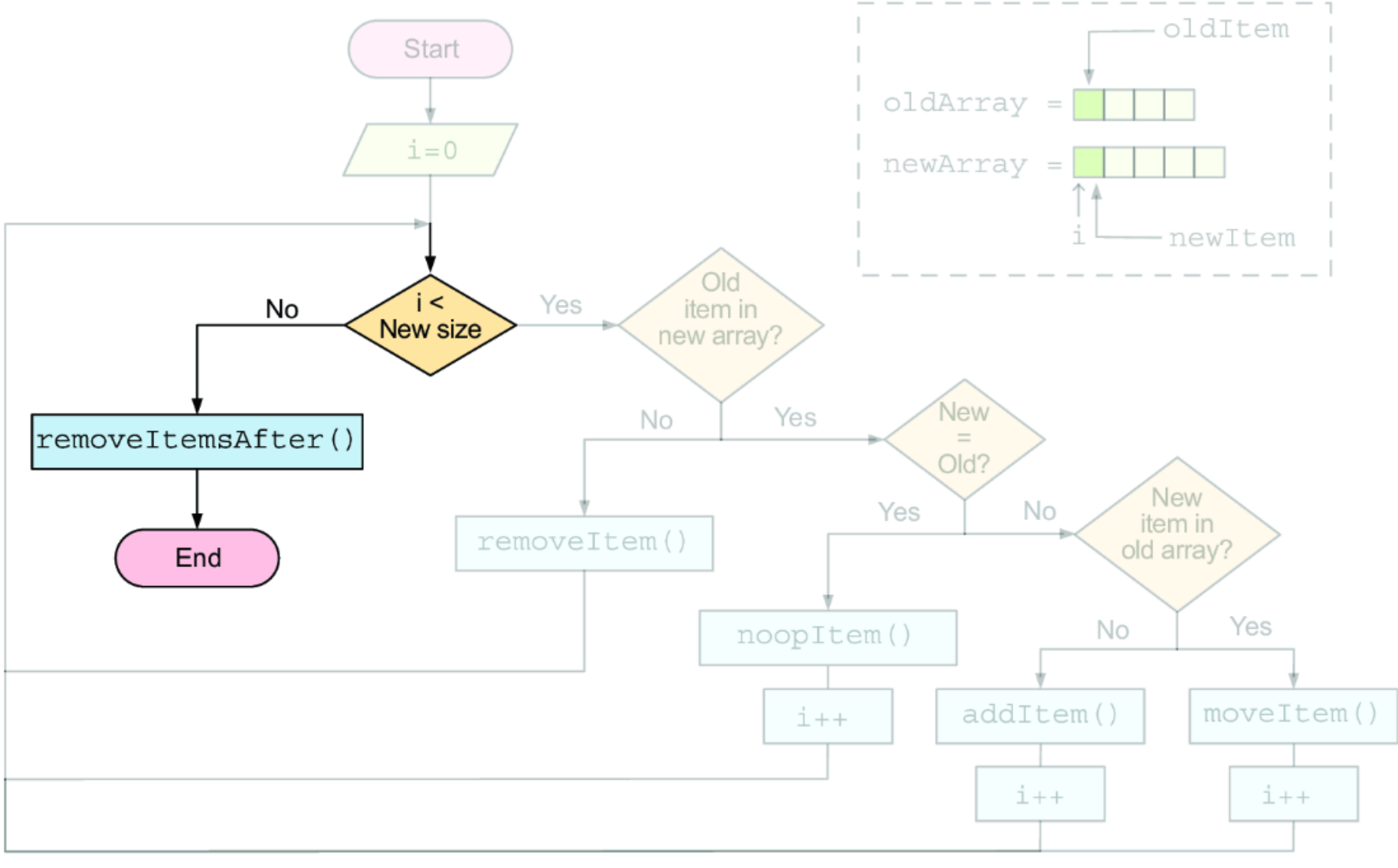
```
    // Extracts the item from the old array\\
    const [_item] = this.#array.splice(fromIndex, 1)
    // Inserts the item into the new position\\
    this.#array.splice(toIndex, 0, _item)

    const [originalIndex] =
    // Extracts the original index from the #originalIndices array\
    this.#originalIndices.splice(fromIndex, 1)
    // Inserts the original index into the new position\\
    this.#originalIndices.splice(toIndex, 0, originalIndex)

    // Returns the move operation\\
    return operation
  }
}
```

```
export function arraysDiffSequence(
  oldArray,
  newArray,
  equalsFn = (a, b) => a === b
) {
  const sequence = []
  const array = new ArrayWithOriginalIndices(oldArray, equalsFn)
```

```
for (let index = 0; index < newArray.length; index++) {
  if (array.isRemoval(index, newArray)) {
    sequence.push(array.removeItem(index))
    index--
    continue
  }

  if (array.isNoop(index, newArray)) {
    sequence.push(array.noopItem(index))
    continue
  }
```

```
    const item = newArray[index]

    if (array.isAddition(item, index)) {
      sequence.push(array.addItem(item, index))
      continue
    }

    sequence.push(array.moveItem(item, index))
  }

 // TODO: remove extra items

  return sequence
}
```

REMOVING THE OUTSTANDING ITEMS



- Remove all items past the current index (which is the length of the new array) from the old array.

```
class ArrayWithOriginalIndices {
 // --snip-- //
  removeItemsAfter(index) {
    const operations = []

    // Keeps removing items while the old array is longer than the 
    while (this.length > index) {
      // Adds the removal operation to the array\\
      operations.push(this.removeItem(index))
    }

    // Returns the removal operations\\
    return operations
  }
}
```

```
export function arraysDiffSequence(
  oldArray,
  newArray,
  equalsFn = (a, b) => a === b
) {
  const sequence = []
  const array = new ArrayWithOriginalIndices(oldArray, equalsFn)

  for (let index = 0; index < newArray.length; index++) {
    if (array.isRemoval(index, newArray)) {
      sequence.push(array.removeItem(index))
      index--
      continue
    }
```

```
    if (array.isNoop(index, newArray)) {
      sequence.push(array.noopItem(index))
      continue
    }

    const item = newArray[index]

    if (array.isAddition(item, index)) {
      sequence.push(array.addItem(item, index))
      continue
    }
    sequence.push(array.moveItem(item, index))
  }
```

```
  sequence.push(...array.removeItemsAfter(newArray.length))

  return sequence
}
```

SUMMARY

- The reconciliation algorithm has two main steps:
  diffing
    (finding the differences between two virtual trees)
  patching
    (applying the differences to the real DOM).

- Diffing two virtual trees to find their differences boils down to solving three problems:
  - finding the differences between two objects,
  - finding the differences between two arrays,
  - finding a sequence of operations that can be applied to an array to transform it into another array.